

UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
Facultatea de
Electronică, Telecomunicații și Tehnologia Informației

Proiect de diplomă

Coordonator științific:

Conf. Dr. Ing. Zbancioc Marius-Dan

Absolvent:

Coța Emilia Elena

IAȘI

2026



Sistem embedded de monitorizare a plantelor prin analiza parametrilor de mediu și procesare de imagini

Coordonator științific,

Conf. Dr. Ing. Zbancioc Marius-Dan

Absolvent,

Coța Emilia Elena

IAȘI

2026

Rezumat

Monitorizarea continuă a plantelor de apartament presupune urmărirea mai multor condiții de mediu: nivelul de umiditate al solului, temperatura, umiditatea aerului și cantitatea de lumină. Neglijarea oricăruia dintre acești factori poate duce la o deteriorare rapidă a plantei. Lucrarea prezintă un sistem embedded de monitorizare bazat pe senzori și pe prelucrarea imaginii, care simplifică acest proces printr-o soluție integrată și furnizează date în timp real.

Sistemul utilizează un microcontroler ESP32 conectat la trei senzori specializați: DHT22 pentru temperatura și umiditatea aerului, un senzor capacitiv pentru umiditatea solului și BH1750 pentru luminozitate. Datele colectate sunt transmise prin protocolul MQTT către platforma Node-RED, unde un algoritm de decizie le procesează și generează recomandări personalizate pentru utilizator. În mod complementar, un script dezvoltat în MATLAB realizează analiza vizuală a stării plantei prin segmentare K-means în spațiul RGB, clasificând pixelii în trei categorii: țesut sănătos (verde), stresat (galben) și uscat (maro). Rezultatele analizei vizuale sunt transmise către Node-RED prin HTTP.

Sistemul a fost testat în condiții reale, obținând valori reprezentative: umiditatea solului de 45,9%, temperatura de 27,3 °C și umiditatea aerului de 70,9%. Analiza imaginilor a indicat 87,1% țesut verde sănătos, 6,3% galben și 6,5% maro, corespunzând statusului „Healthy”. Toate datele sunt vizualizate în timp real printr-un dashboard Node-RED, care include grafice, o plantă animată cu expresie adaptivă și un asistent virtual cu recomandări contextuale.

Rezultatele demonstrează că sistemul permite monitorizarea completă și fiabilă a plantelor de apartament, combinând date senzoriale cu analiza vizuală într-o arhitectură coerentă și scalabilă.

Cuprins

Rezumat	iii
Cuprins	iv
Listă figuri.....	vii
Memoriu justificativ.....	ix
1. Introducere	1
1.1. Prezentarea generală a domeniului IoT	1
1.2. Structura lucrării	3
2. Stadiul actual.....	5
2.1. Sisteme IoT pentru aglicultură inteligentă	5
2.2. Tehnologii și protocoale utilizate în sisteme IoT	7
2.3. Analiza poziției proiectului față de soluțiile existente	8
3. Arhitectura sistemului	10
3.1. Descrierea generală a arhitecturii.....	10
3.2. Componente hardware	11
3.2.1. Microcontroler ESP32.....	11
3.2.2. Senzor de temperatură și umiditate DHT22.....	13
3.2.3. Senzor capacitiv de umiditate a solului.....	15
3.2.4. Senzor de luminozitate BH1750	16
3.2.5. Montajul experimental pe breadboard	17
3.3. Software și protocoale de comunicație	19
3.3.1. Programarea ESP32(Arduino IDE).....	19
3.3.2. Protocolul MQTT și brokerul Mosquitto	20
3.3.3. Platforma Node-RED	22
4. Analiza stării plantei prin procesare de imagini(MATLAB)	25
4.1. Metoda de segmentare K-means în spațiul RGB	25
4.2. Calculul procentajelor de sănătate a plantei	29

4.3. Integrarea cu Node-RED prin HTTP	31
5. Algoritmul de decizie și sistemul de recomandări.....	32
5.1. Praguri și criterii utilizate	32
5.2. Logica de combinare a datelor din senzori și analiza de imagine	34
5.3. Generarea recomandărilor pentru utilizator.....	36
6. Interfața utilizator – Dashboard Node-RED	39
6.1. Structura și organizarea dashboard-ului	39
6.2. Elemente vizuale și widget-uri	41
7. Testare și rezultate	44
7.1. Scenarii de testare	44
7.2. Rezultate obținute și interpretare	45
7.3. Limitări și probleme întâlnite	47
8. Concluzii și direcții viitoare de dezvoltare	49
8.1. Concluzii generale	49
8.2. Direcții viitoare de dezvoltare	50
9. Bibliografie.....	53
10. Anexe.....	54

Listă figuri

Figura 1.1. Arhitectura unui sistem IoT bazat pe ESP32 și Node-RED.....	2
Figura 2.1. Arhitectura generală a unui sistem robotic IoT utilizat în agricultură.	6
Figura 2.2. Diagrama de implementare hardware a platformei mobile autonome.....	6
Figura 2.3. Diagrama arhitecturală a sistemului IoT	7
Figura 2.4. Arhitectura bloc a sistemului de detecție bazat pe Deep Learning.	7
Figura 3.1. Schema bloc a arhitecturii sistemului.	11
Figura 3.2. Modulul ESP32.	11
Figura 3.3. Modulul ESP32 cu pinout-ul complet.....	12
Figura 3.4. Senzorul DHT22	12
Figura 3.5. Senzor capacitiv de umiditate a solului.....	15
Figura 3.6. Senzorul de luminozitate BH1750	16
Figura 3.7. Montajul experimental al sistemului pe breadboard	17
Figura 3.8. Montajul experimental al sistemului pe breadboard – ESP32 cu conexiunile senzorilor	18
Figura 3.9. Montajul experimental al sistemului pe breadboard împreună cu planta în suport	18
Figura 3.10. Schema electrică de interconectare a senzorilor de achiziție cu ESP32 în mediul EasyEDA	19
Figura 3.11. Includerea și definirea pinilor și a MQTT din firmware-ul ESP32	19
Figura 3.12. Funcția setup() pentru conectarea Wi-Fi și MQTT.....	20
Figura 3.13. Publicarea datelor senzorilor pe topicurile MQTT din firmware-ul ESP32	20
Figura 3.14. Adresa IP a calculatorului gazdă, rezultatul comenzii ipconfig.....	22
Figura 3.15. Monitorizarea în timp real a parametrilor de mediu prin conexiunea MQTT.....	22
Figura 3.16. Flow-ul Node-RED al sistemului.....	23
Figura 3.17. Dashboard-ul interactiv Node-RED	23
Figura 3.18. Serial Monitor cu valorile trimise ulterior spre Node-RED.....	24
Figura 4.1. Imaginea originală a plantei supusă analizei vizuale	28
Figura 4.2. Rezultatul segmentării prin algoritmul K-means cu 6 clustere (fundal eliminat din mască).....	29
Figura 4.3. Rezultatele analizei cromatice din MATLAB	30
Figura 4.4. Structura fluxului de recepție asincronă HTTP POST în Node-RED.....	31
Figura 5.1. Condițiile algoritmului de decizie implementat în Node-RED.....	34
Figura 5.2. Recepția și stocarea datelor de la senzori prin topicuri MQTT	35

Figura 5.3. Integrarea rezultatelor analizei vizuale din MATLAB în algoritmul de decizie ...	35
Figura 5.4. Interfața dashboard în stare critică.....	36
Figura 5.5. Componenta Virtual Assistant în stare optimă.....	38
Figura 6.1. Interfața de configurare a nodului ui_template și inițializarea variabilelor pentru scara logaritmică în Node-RED	43
Figura 6.2. Aspectul general al dashboard-ului interactiv în starea de avertizare	43
Figura 7.1. Rezultatele numerice ale analizei cromatice foliare generate în MATLAB	46
Figura 7.2. Componenta Virtual Assistant în starea de avertizare.....	47

Memoriu justificativ

Conceptul de Internet al Lucrurilor (IoT – Internet of Things) a schimbat semnificativ modul în care interacționăm cu mediul. Această evoluție a dus la trecerea rapidă de la automatizări industriale rigide la sisteme inteligente și adaptabile pentru locuințe. În acest context, monitorizarea și optimizarea microclimatului pentru plante au devenit o temă de cercetare foarte actuală. Un articol recent arată că sistemele complet automatizate, care folosesc mai mulți senzori și procesarea imaginilor, pot detecta din timp stresul hidric și pot permite irigarea zonelor specifice. Astfel, se depășesc limitele metodelor tradiționale, în care factori importanți precum umiditatea solului, temperatura, umiditatea aerului sau lumina sunt adesea ignorați, ceea ce poate duce la deteriorarea rapidă a plantelor. Proiectul de față propune o soluție integrată care elimină dependența de platforme comerciale închise și costisitoare, oferind o arhitectură open-source.

Proiectul este relevant deoarece combină direct metodele de achiziție senzorială cu tehnici avansate de analiză vizuală, oferind o abordare hibridă care aduce originalitate față de soluțiile hardware clasice. Nodul de achiziție de la marginea rețelei folosește microcontrolerul ESP32, la care sunt conectați trei senzori: DHT22 pentru temperatura și umiditatea aerului, un senzor capacitiv pentru umiditatea solului și senzorul digital BH1750 pentru intensitatea luminii. Datele colectate sunt transmise asincron prin protocolul MQTT către brokerul local Mosquitto, evitând blocarea execuției codului pe placa de dezvoltare. Noutatea constă în integrarea unui script de procesare a imaginilor, dezvoltat în MATLAB, care realizează segmentarea spectrală prin algoritmul K-means în spațiul de culoare RGB. Algoritmul clasifică pixelii imaginii în trei categorii importante pentru diagnosticare: țesut sănătos (verde), stresat (galben) și uscat (maro), iar rezultatele analizei vizuale sunt transmise serverului central prin protocolul HTTP.

Scopul principal al lucrării a fost proiectarea, implementarea și validarea experimentală a acestui ecosistem pentru monitorizarea microclimatului. Obiectivele au inclus calibrarea componentelor hardware, configurarea comunicației prin ESP32, maparea topicurilor MQTT, implementarea algoritmului de segmentare în MATLAB și dezvoltarea logicii de decizie în Node-RED.

Validarea experimentală a prototipului în condiții reale a demonstrat fiabilitatea arhitecturii propuse. Au fost obținute valori reprezentative: o umiditate a solului de 52,5%, o temperatură ambientală de 21,6 °C și o umiditate a aerului de 56,1%. Concomitent, analiza vizuală realizată prin scriptul MATLAB a indicat o distribuție de 87,12% țesut verde sănătos, 6,33% țesut galben și 6,55% țesut maro, confirmând statusul general al plantei, sănătos („Healthy”). Toate aceste rezultate sunt centralizate și vizualizate în timp real printr-un

dashboard grafic interactiv în Node-RED. Acesta include grafice, o plantă animată cu expresii adaptive și un asistent virtual care generează recomandări contextuale de acțiune. Întreaga structură a proiectului, de la interconectarea fizică pe breadboard la scrierea codului în Arduino IDE, configurarea middleware-ului și scrierea elementelor dinamice din nodurile ui_template, a fost realizată integral de autor. Dezvoltarea acestei lucrări de diplomă a avut un impact profesional major, facilitând aprofundarea cunoștințelor în sisteme încorporate, rețele de calculatoare și procesarea imaginii, competențe fundamentale pentru un inginer din domeniul electronicii aplicate.

1. Introducere

Sectorul agricol și monitorizarea plantelor, inclusiv a celor de apartament, se confruntă cu provocări care necesită optimizarea resurselor și un management eficient. Metodele tradiționale se bazează pe observație manuală și pe îngrijire periodică, ceea ce duce adesea la o utilizare ineficientă a resurselor, la o creștere inconsecventă și la o reacție întârziată la schimbările din mediu [1]. Neglijarea unor factori critici de microclimat, precum nivelul de umiditate al solului, temperatura, umiditatea aerului și cantitatea de lumină, poate duce la deteriorarea rapidă a plantelor [2].

Deși există soluții comerciale pentru monitorizare, acestea sunt adesea costisitoare, închise sau se concentrează pe un singur parametru. Arhitecturile open-source bazate pe ESP32 și Node-RED oferă o alternativă viabilă și rentabilă față de platformele comerciale închise. Lucrarea de față propune un sistem embedded, integrat, open-source și accesibil, care combină achiziția de date senzoriale cu tehnici avansate de analiză vizuală.

Sistemul utilizează un microcontroler ESP32 și o rețea de senzori, DHT22 pentru temperatura și umiditatea aerului, un senzor capacitiv pentru umiditatea solului și BH1750 pentru intensitatea luminoasă, pentru a colecta date în timp real. Elementul de noutate este adus de implementarea unei analize vizuale a stării plantei, prin segmentare K-means în mediul MATLAB, care clasifică starea țesutului în trei categorii: verde (sănătos), galben (stresat) și maro (uscat) [3]. Datele sunt transmise prin protocolul MQTT către platforma Node-RED, unde un algoritm de decizie oferă asistență și recomandări personalizate utilizatorului.

1.1. Prezentarea generală a domeniului IoT

Internet of Things (IoT) reprezintă o rețea de dispozitive fizice încorporate cu componente electronice, senzori, actuatori, software și conectivitate, permițând acestor dispozitive să se conecteze, să colecteze și să schimbe date, luând totodată decizii inteligente [4]. Această tehnologie a revoluționat multiple domenii, asigurând trecerea de la automatizări rigide la sisteme proactive și adaptative. Conform raportului Cisco Annual Internet Report (2023), se estimează că până în 2030 vor fi conectate peste 29 de miliarde de dispozitive IoT la nivel global [5], ceea ce subliniază amploarea și impactul acestei paradigme tehnologice. Sistemele IoT pot fi structurate pe mai multe niveluri arhitecturale, fiecare cu un rol distinct în fluxul de date:

- Nivelul fizic / de percepție (Sensing Layer) este responsabil de colectarea datelor din mediul fizic. În cadrul proiectului de față, acest nivel este acoperit de microcontrolerul

1.1. Prezentarea generală a domeniului IoT

ESP32, un SoC (System on Chip) cu consum redus de energie și conectivitate Wi-Fi integrată, ideal pentru aplicații IoT [6], precum și de senzorii DHT22 și BH1750, precum și de senzorul capacitiv de umiditate a solului.

- Nivelul de rețea (Network Layer) asigură transmiterea datelor prin intermediul rețelelor wireless. În cazul sistemului propus, comunicația se realizează prin Wi-Fi, utilizând protocolul MQTT (Message Queuing Telemetry Transport), ales pentru lățimea de bandă redusă, fiabilitatea și eficiența sa în rețelele locale [2].
- Nivelul middleware-ului și cel de aplicație (Application Layer) facilitează interfața dintre dispozitive și utilizator. Platforma Node-RED funcționează ca un server IoT edge care preia datele, aplică logica de decizie și le prezintă într-un dashboard interactiv accesibil din browser.

O tendință modernă în domeniul IoT este extinderea capabilităților de monitorizare senzorială prin integrarea inteligenței artificiale și a procesării imaginii (Computer Vision). Studii recente demonstrează că sistemele IoT, combinate cu procesarea imaginilor, pot atinge o acuratețe de peste 90% în detectarea timpurie a bolilor foliare și a stresului hidric [7]. Prin analiza imaginilor realizată cu algoritmul de segmentare K-means în MATLAB, sistemul propus poate detecta modificări vizibile ale stării plantei care nu ar fi sesizabile exclusiv prin senzorii de mediu, oferind astfel o diagnoză mai completă și mai precisă [3]. În Figura 1.1 este ilustrată arhitectura generală a unui sistem IoT structurat pe niveluri, similară cu arhitectura adoptată în cadrul proiectului de față.

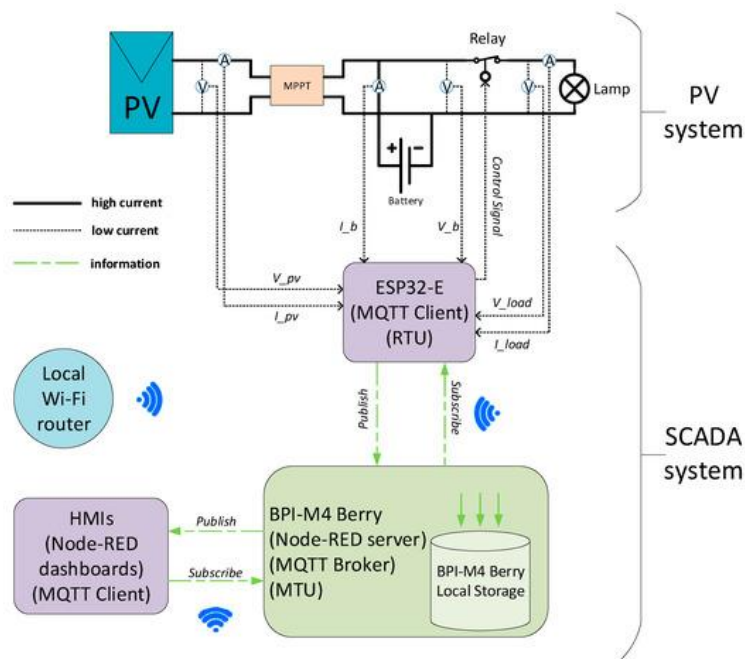


Figura 1.1. Arhitectura unui sistem IoT bazat pe ESP32 și Node-RED

1.2. Structura lucrării

Lucrarea este organizată în opt capitole, fiecare acoperind un aspect distinct al proiectului.

- Capitolul 2 analizează sistemele IoT existente pentru agricultura inteligentă și tehnologiile aferente, evidențiind poziția și noutatea proiectului de față comparativ cu soluțiile comerciale și academice identificate în literatura de specialitate.
- Capitolul 3 detaliază designul hardware și software al sistemului. Sunt prezentate componentele fizice: microcontrolerul ESP32, senzorii DHT22 și BH1750, precum și senzorul capacitiv de umiditate a solului, interconectarea lor pe breadboard, precum și protocoalele de comunicație utilizate: MQTT, brokerul Mosquitto și platforma Node-RED.
- Capitolul 4 explică metodologia de evaluare vizuală a sănătății plantei prin segmentarea K-means în spațiul RGB, implementată în mediul MATLAB, precum și modul de integrare a rezultatelor cu platforma Node-RED prin protocolul HTTP.
- Capitolul 5 prezintă logica de combinare a datelor achiziționate de senzori cu rezultatele analizei de imagine, pragurile de alertă utilizate și mecanismul de generare a recomandărilor personalizate pentru utilizator.
- Capitolul 6 descrie designul interfeței grafice interactive dezvoltate în Node-RED, elementele vizuale și organizarea informației: grafice, asistent virtual, plantă animată cu expresie adaptivă, pentru a oferi feedback în timp real.
- Capitolul 7 expune scenariile de testare în condiții reale ale prototipului, rezultatele obținute în urma funcționării senzorilor și a analizei imaginii, precum și limitările și problemele întâlnite pe parcursul dezvoltării.
- Capitolul 8 sintetizează realizările proiectului, evaluează gradul de îndeplinire a obiectivelor propuse și identifică direcții viitoare de dezvoltare și de extindere a arhitecturii.

2. Stadiul actual

Tranziția către automatizarea completă a sectorului agricol și horticola este pe deplin justificată de limitările severe ale metodelor tradiționale de detectare a bolilor sau a stresului fiziologic la plante. În mod clasic, aceste proceduri se bazează masiv pe inspecția vizuală manuală realizată de experți umani sau agronomi, un proces recunoscut în literatura de specialitate ca fiind un mare consumator de timp, cu o puternică componentă subiectivă și predispus la erori umane în eșantionare.

2.1. *Sisteme IoT pentru aglicultură inteligentă*

Prin integrarea dispozitivelor IoT, a senzorilor, a dronelor și a inteligenței artificiale, agricultura este revoluționată, permițând monitorizarea constantă a nivelului de umiditate al solului, a temperaturii și a stării generale de sănătate a plantelor.

Colectarea continuă a indicatorilor de microclimat, corelată cu sisteme inteligente de analiză, permite detectarea timpurie a stresului hidric sau termic înainte ca simptomele să devină vizibile macroscopic la nivelul aparatului foliar. Astfel, prin interconectarea rețelelor distribuite de senzori cu algoritmi de decizie capabili de tratare adaptivă, se asigură o gestiune optimizată a resurselor de apă și de nutrienți [9].

Sisteme mobile cu camere video (roboți autonomi în teren/sere): O inovație majoră prezentată în literatura recentă o reprezintă sistemele robotizate autonome alimentate cu energie solară, concepute pentru a naviga printre rândurile de plante (în câmp sau în sere) [8].

Aceste sisteme sunt echipate cu camere de înaltă rezoluție și senzori de mediu (umiditate sol, temperatură). Robotul se deplasează autonom, captează imagini ale frunzelor la intervale regulate și le procesează în timp real la nivelul echipamentului (edge computing) pentru a detecta bolile [9], trimițând alerte utilizatorului cu locația exactă a plantei afectate.

Această abordare bazată pe noduri mobile de scanare reduce semnificativ volumul de date transmise prin rețea către serverele cloud, deoarece imaginile brute, de mari dimensiuni, sunt analizate local. Spre infrastructura centrală de monitorizare sunt expediate doar metadatele, vectorii de caracteristici sau flagurile de alertă care conțin coordonatele spațiale ale vectorului de infecție. Integrarea tehnicilor de procesare locală pe platforme mobile deschide noi direcții în dezvoltarea serelor inteligente, demonstrând că o arhitectură hardware optimizată poate compensa lipsa unei conexiuni permanente la internet, fără a compromite rigurozitatea procesului de monitorizare a stării de sănătate a plantelor.

2.1. Sisteme IoT pentru aglicultură inteligentă

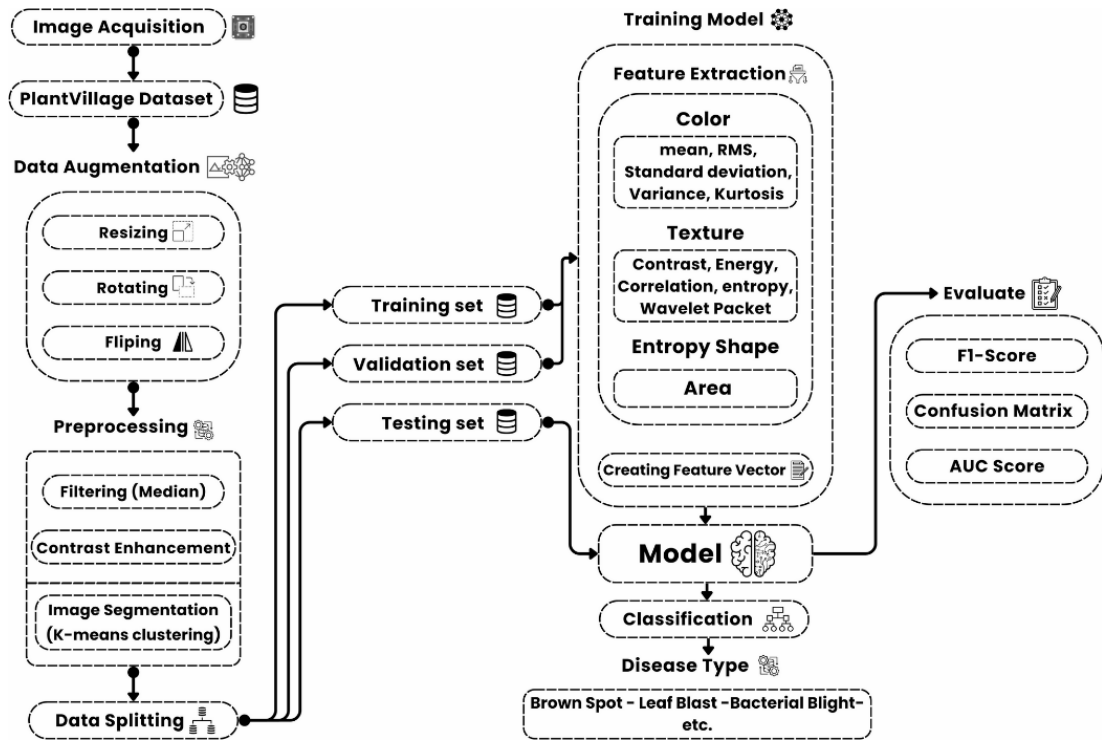


Figura 2.1. Arhitectura generală a unui sistem robotic IoT utilizat în agricultură [8]

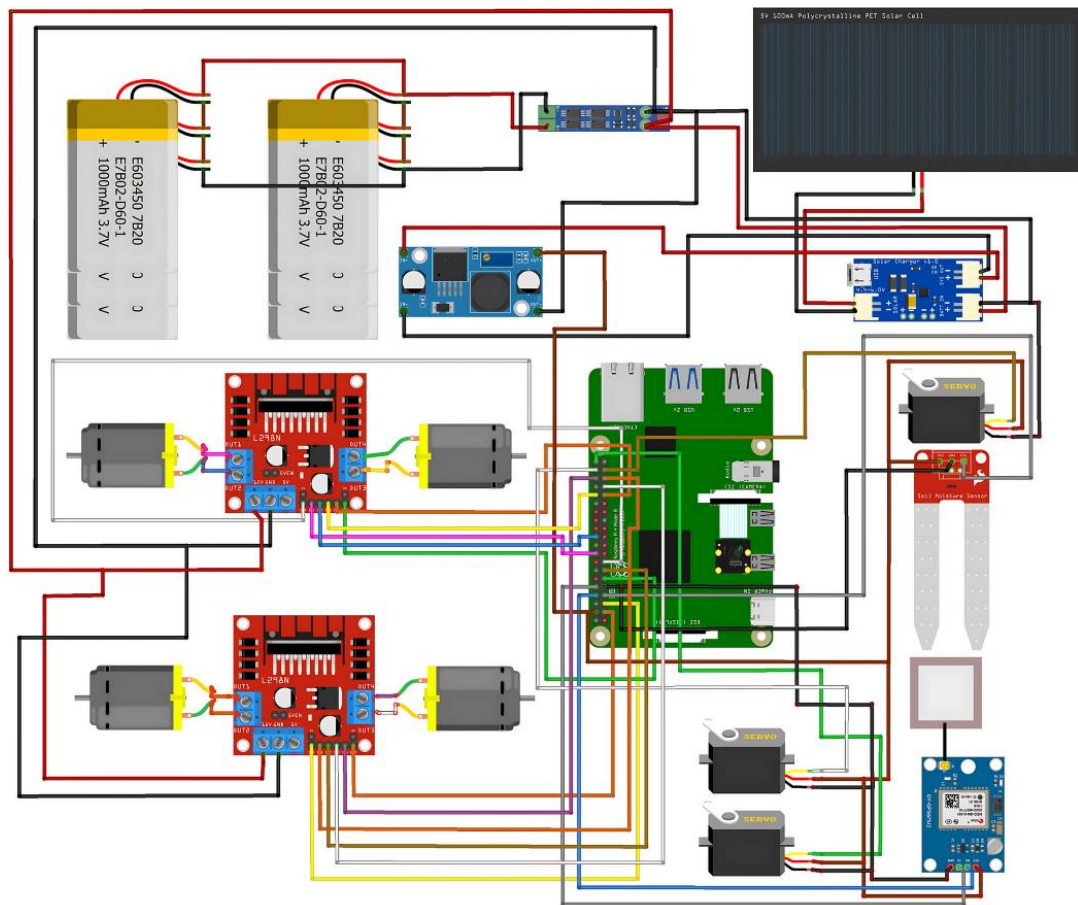


Figura 2.2. Diagrama de implementare hardware a platformei mobile autonome (panou solar, motoare, senzori si unitate de calcul) [8]

2.2. Tehnologii și protocoale utilizate în sisteme IoT

Rețele Neuronale și Deep Learning pentru sănătatea plantelor: Pentru detectarea vizuală a bolilor direct din imaginile frunzelor (în special pentru identificarea petelor specifice diverselor infecții), se utilizează pe scară largă Rețelele Neuronale Convoluționale (CNN), precum arhitecturile VGG, ResNet, DenseNet, Inception sau YOLO [10]. Aceste rețele sunt antrenate pe seturi masive de date (cum ar fi PlantVillage sau PlantDoc, care conțin zeci de mii de imagini) pentru a extrage automat caracteristicile bolilor.

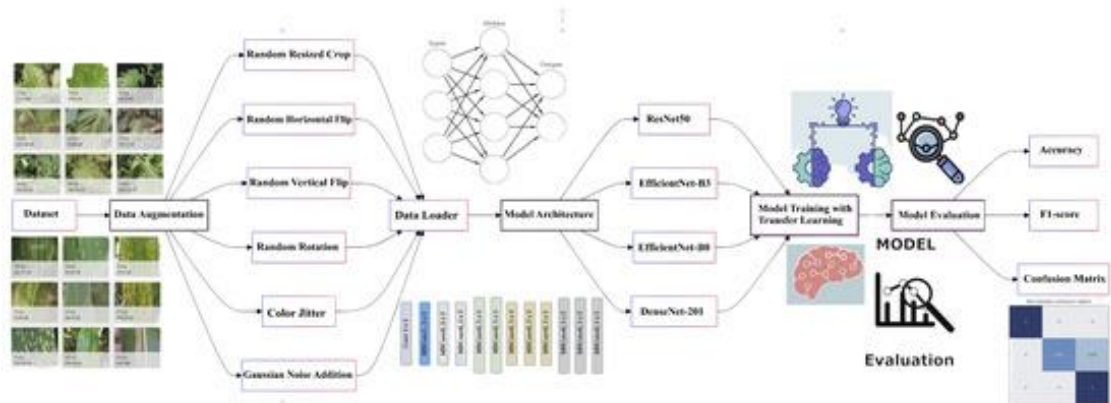


Figura 2.3. Diagrama arhitecturală a sistemului IoT [11]

Detecția după pete de pe frunze: Modelele antrenate pot recunoaște extrem de precis boli care se manifestă prin pete (leaf spots), cum ar fi pătarea cenușie a frunzelor de porumb (Corn grey leaf spot), rugina mărului (Apple leaf rust), mana timpurie sau târzie a tomatelor (Tomato early/late blight) sau virusul mozaicului [10]. Modelele precum EfficientNet sau Inception v3 au atins o acuratețe de 90–99% în clasificarea acestor boli direct din imagini cu frunze [11].

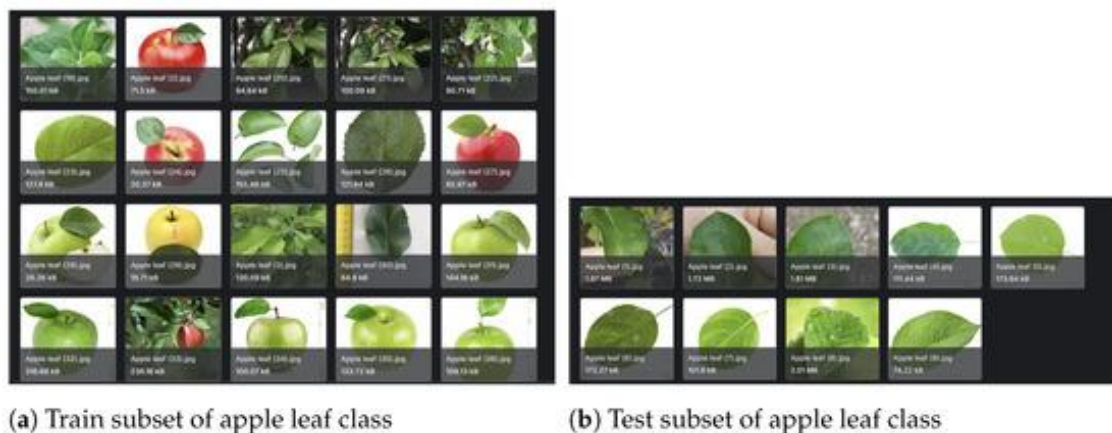


Figura 2.4. Arhitectura bloc a sistemului de detecție bazat pe Deep Learning

Protocoale și procesare Edge (la fața locului): Pentru a reduce latența și dependența de conexiuni internet rapide, sistemele moderne utilizează un procesor local, precum Raspberry

2.3. Analiza poziției proiectului față de soluțiile existente

Pi, care rulează modele AI comprimate (prin TensorFlow Lite). Datele senzorilor de mediu (umiditate, temperatură) și alertele de boală sunt transmise folosind protocolul MQTT, ales pentru eficiența sa, către o interfață (dashboard) accesibilă pe telefon sau pe web.

2.3. Analiza poziției proiectului față de soluțiile existente

Limitările soluțiilor actuale: Deși modelele de tip Deep Learning (CNN) sunt extrem de precise, necesită adesea resurse computaționale ridicate și hardware costisitor, fiind inaccesibile pentru fermierii mici sau pasionații de plante de apartament. Mai mult, multe modele sunt antrenate doar pe seturi de date preluate în condiții de laborator (fundal uniform, lumină perfectă), iar atunci când sunt testate în lumea reală, acuratețea lor scade dramatic din cauza fundalurilor complexe sau a iluminării slabe. De asemenea, soluțiile bazate pe drone (UAV) au o autonomie redusă a bateriei și pot fi limitate de condițiile meteo. Multe sisteme fac exclusiv analiză vizuală, ignorând datele de microclimat, ceea ce limitează diagnosticul corect.

Avantajele și poziția proiectului propus: Proiectul de față abordează direct problema fragmentării funcționalităților observată în literatura de specialitate. Prin combinarea achiziției de date de mediu (senzor de sol, luminozitate, temperatură DHT22/ESP32) cu analiza vizuală a stării plantei, prin procesarea de imagini (segmentare K-means în MATLAB, pentru a evita costurile hardware masive ale unui sistem complet bazat pe Deep Learning), sistemul propus este mult mai eficient din punct de vedere al costurilor, ușor de implementat și integrat. Folosind Node-RED și MQTT, proiectul oferă funcționalitatea exactă a unui asistent virtual în timp real, care lipsește din multe sisteme academice statice. Astfel, proiectul propune o arhitectură IoT de tip "edge-to-cloud", accesibilă, locală, dar complet funcțională pentru monitorizarea sănătății plantelor.

Poziționarea strategică a actualului proiect în raport cu soluțiile documentate în literatura de specialitate evidențiază o orientare clară către optimizarea raportului dintre complexitatea computațională și costul de implementare a hardware-ului. Majoritatea sistemelor academice actuale tind să migreze exclusiv către rețele neuronale convoluționale de mari dimensiuni, ignorând faptul că aplicarea acestora necesită unități de procesare grafică masive, inaccesibile pentru utilizatorul convențional sau pentru sistemele embedded cu resurse limitate.

Prin adoptarea algoritmului nesupervizat K-means în mediul MATLAB și decuplarea fazei de segmentare cromatică de procesul de achiziție hardware de pe ESP32, proiectul propus demonstrează o scalabilitate superioară. Sistemul folosește un protocol de transport extrem de ușor (MQTT), apelând la analiza vizuală prin HTTP POST doar la intervale stabilite sau la

cerere, ceea ce conservă lăţimea de bandă a reţelei locale şi previne suprasolicitarea procesorului periferic.

De asemenea, spre deosebire de platformele comerciale rigide, care obligă utilizatorul să îşi stocheze datele pe servere cloud proprietare sau externe, arhitectura hibridă dezvoltată în această lucrare oferă independenţă totală prin utilizarea unui middleware local (Node-RED). Acest aspect nu doar că elimină costurile recurente de administrare, ci garantează şi confidenţialitatea absolută a datelor transmise, precum şi flexibilitatea totală în reconfigurarea ulterioară a interfeţei grafice sau a arborilor logici de decizie, în funcţie de evoluţia specifică a culturii monitorizate.

3. Arhitectura sistemului

Implementarea unui sistem inteligent capabil de monitorizare hibridă, prin fuziunea datelor senzoriale cu analiza spectrală a imaginii, necesită proiectarea unei infrastructuri robuste și bine structurate. Obiectivul acestui capitol este de a detalia arhitectura generală a sistemului propus, evidențiind modul în care componentele hardware, subsistemele software și protocoalele de comunicație interacționează pentru a asigura un flux continuu și asincron de date.

În cadrul secțiunilor următoare se analizează topologia modulară pe niveluri, specificațiile tehnice ale microcontrolerului central și ale senzorilor alocați procesului de achiziție, precum și detaliile realizării montajului experimental. De asemenea, este documentată logica de programare a nodului periferic și se justifică alegerea arhitecturii de rețea, punând accent pe eficiența protocolului de transport și pe integrarea middleware-ului în platformele de procesare. Prin această abordare sistemică se fundamentează suportul fizic și logic pe baza căruia se realizează diagnoza în timp real a stării de sănătate a plantelor.

3.1. Descrierea generală a arhitecturii

Arhitectura sistemului propus urmează un model modular structurat pe trei niveluri interconectate, specific aplicațiilor IoT:

- **Nivelul de percepție (Sensing Layer):** Reprezintă stratul fizic, responsabil de interacțiunea directă cu mediul. Aici regăsim componentele hardware destinate achiziției de date: senzorii pentru monitorizarea microclimatului plantei (DHT22), senzorul capacitiv de umiditate a solului și senzorul BH1750.
- **Nivelul de procesare și rețea (Processing & Network Layer):** Centralizează datele la marginea rețelei (edge computing). Acest nivel este gestionat de microcontrolerul ESP32, care citește valorile analogice și digitale, le prelucrează local și le transmite wireless folosind rețeaua Wi-Fi locală.
- **Nivelul aplicației (Application Layer):** Cuprinde logica de pe server, platforma Node-RED pentru asistență și vizualizare, precum și scriptul MATLAB care gestionează componenta de analiză vizuală. Datele circulă fluent între aceste niveluri, asigurând o comunicare asincronă, fără blocare.

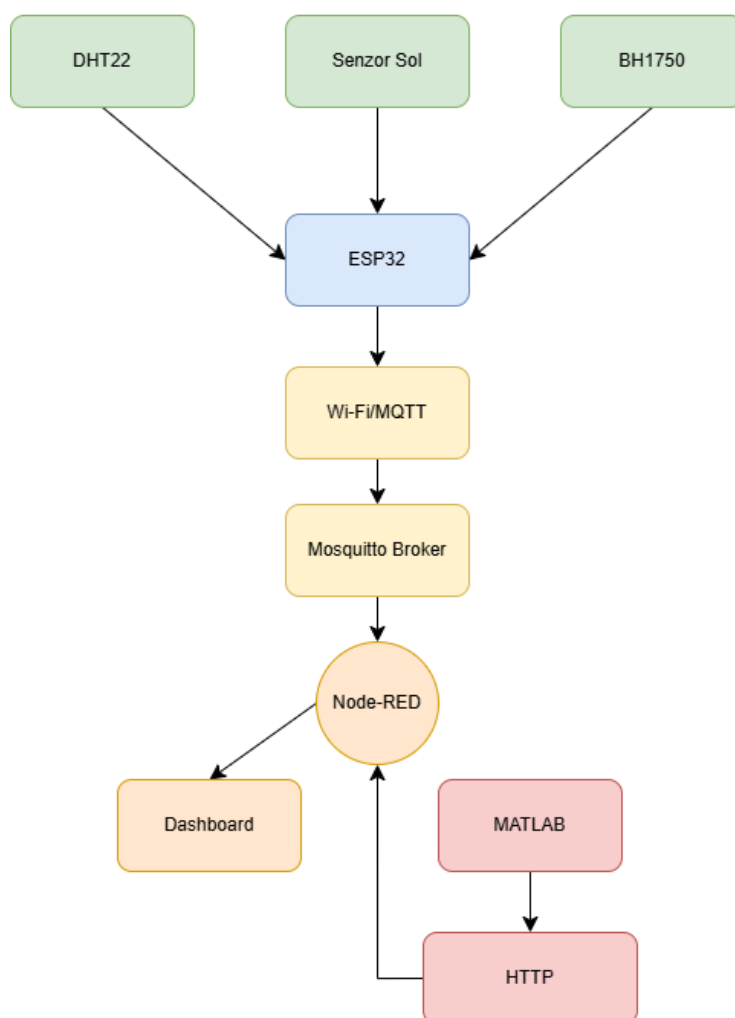


Figura 3.1. Schema bloc a arhitecturii sistemului

3.2. Componente hardware

3.2.1. Microcontroler ESP32

ESP32 acționează ca unitate centrală de procesare a sistemului. Este un SoC (System on Chip) cu consum redus de energie și performanță ridicată, echipat cu un microprocesor dual-core Tensilica Xtensa LX6 cu o frecvență de până la 240 MHz. Integrarea nativă a conectivității Wi-Fi și Bluetooth facilitează transmiterea directă a pachetelor de date către brokerul de mesaje. De asemenea, microcontrolerul dispune de multiple interfețe GPIO (General Purpose Input/Output), care permit conectarea diverselor tipuri de senzori, având totodată moduri de "deep sleep" pentru o eficiență energetică sporită [12].

Din punct de vedere hardware, cipul ESP32 dispune de o arhitectură internă optimizată pentru procesarea concurentă, având două nuclee independente capabile de procesare simetrică. Această structură permite partajarea eficientă a resurselor hardware: în timp ce un nucleu este alocat exclusiv gestionării stivei de comunicație fără fir și menținerii conexiunii active la

3.2. Componente hardware

rețeaua locală, cel de-al doilea nucleu rulează ciclic rutina de achiziție a datelor de la senzori, de parsare a valorilor brute și de formatare a stringurilor de transmisie.

Un aspect critic în stabilitatea pe termen lung a sistemului este gestionarea memoriei volatile SRAM, utilizată pentru maparea dinamică a stivei de rețea Wi-Fi și a bufferelor de mesaje MQTT. Deoarece protocolul de rețea Wi-Fi și biblioteca clientului de mesaje folosesc buffere dinamice pentru stocarea temporară a pachetelor, s-a acordat o atenție deosebită evitării fragmentării memoriei. Prin definirea unor variabile globale cu dimensiune fixă pentru stocarea valorilor achiziționate și prin utilizarea unor buffere de caractere prealocate pentru generarea mesajelor destinate publicării, execuția firmware-ului este protejată împotriva erorilor de tip depășire de stivă.

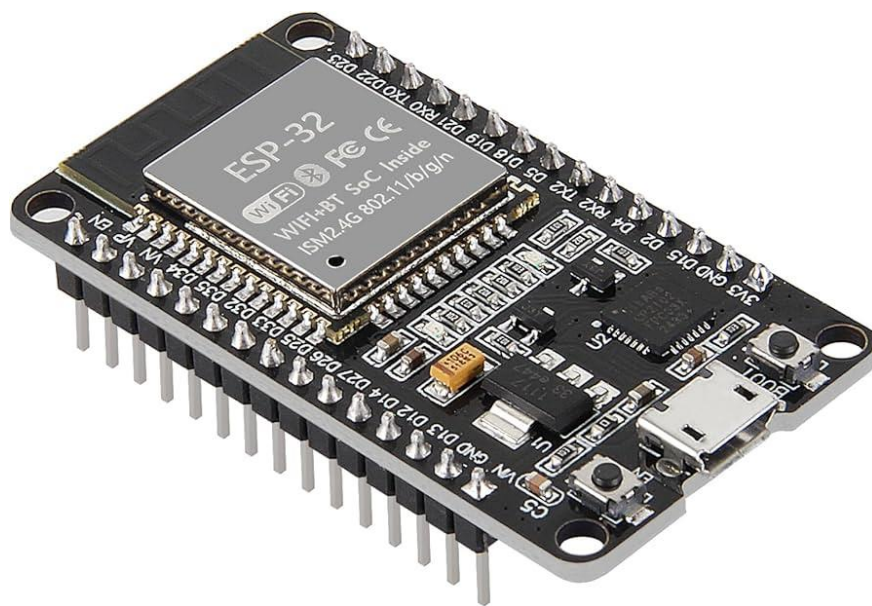


Figura 3.2. Modulul ESP32

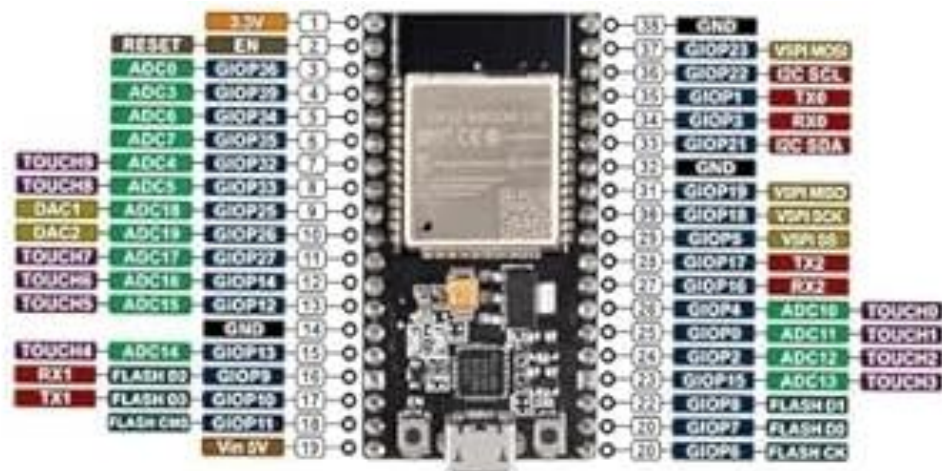


Figura 3.3. Modulul ESP32 cu pinout-ul complet

Pentru a centraliza interconectările fizice realizate în cadrul montajului experimental, în Tabelul 3.1 sunt prezentați pinii GPIO ai microcontrolerului ESP32 utilizați în proiect, împreună cu funcția și componenta asociate fiecăruia.

Tabelul 3.1 Pinii GPIO ai ESP32 utilizați în sistemul de monitorizare

Pin GPIO	Funcție	Componentă conectată
GPIO23	Digital Input(One-Wire)	Senzor DHT22 – linie de date
GPIO 35	ADC15 – Analog Input	Senzor capacitiv de umiditate sol
GPIO21	I2C – SCL	Senzor BH1750 – linie de ceas
GPIO22	I2C – SDA	Senzor BH1750 – linie de date
3.3V	Alimentare	VCC comun
GND	Masă	GND comun

Un aspect important al arhitecturii de pini alese îl reprezintă compatibilitatea GPIO35 cu convertorul analog-digital intern al ESP32. Pinul GPIO35 face parte din grupul ADC1, singurul canal ADC funcțional atunci când conectivitatea Wi-Fi este activă. Grupul ADC2 al ESP32 prezintă o limitare hardware cunoscută: pinii săi sunt partajați cu driverul Wi-Fi intern, iar activarea acestuia dezactivează complet ADC2, făcând imposibilă citirea semnalului analogic al senzorului de sol. Prin alocarea deliberată a senzorului capacitiv pe un pin din grupul ADC1, firmware-ul poate achiziționa simultan datele analogice ale senzorului și poate menține activ protocolul MQTT prin Wi-Fi, fără conflicte hardware.

3.2.2. Senzor de temperatură și umiditate DHT22

Pentru monitorizarea microclimatului aerului se utilizează senzorul digital DHT22. Acesta oferă o acuratețe de $\pm 0,5$ °C pentru temperatură și de $\pm 2\%$ pentru umiditatea relativă a aerului. Comunicarea cu microcontrolerul se face printr-o interfață de tip "single-wire", fiind un dispozitiv fiabil și ușor de integrat în sisteme cu un număr limitat de pini [13].

Principiul de funcționare al senzorului DHT22 se bazează pe două elemente senzor integrate în același modul: un element capacitiv pentru măsurarea umidității și un termistor NTC pentru măsurarea temperaturii. Elementul capacitiv este alcătuit dintr-un substrat polimeric higroscopic plasat între doi electrozi conductivi. Pe măsură ce moleculele de apă din aer sunt absorbite de stratul polimeric, constanta dielectrică a acestuia se modifică proporțional

3.2. Componente hardware

cu concentrația vaporilor, determinând o variație măsurabilă a capacității condensatorului format. Circuitul integrat intern al senzorului convertește această variație capacitivă într-o valoare digitală a umidității relative, exprimată în procente[13].

Comunicația dintre DHT22 și microcontrolerul ESP32 se realizează printr-o interfață serială single-wire, pe un singur pin de date bidirecțional. Protocolul de comunicație este inițiat de microcontroler printr-un semnal START: linia de date este trasă la nivel logic scăzut pentru minimum 1 ms, după care este eliberată și menținută la nivel înalt timp de 20–40 μ s. Senzorul răspunde prin confirmarea prezenței sale cu un puls de nivel scăzut de 80 μ s, urmat de unul de nivel înalt tot de 80 μ s, după care transmite secvențial cei 40 de biți de date. Fiecare bit este codificat prin durata impulsului de nivel înalt: un puls de aproximativ 26–28 μ s corespunde valorii logice „0”, în timp ce un puls de 70 μ s corespunde valorii „1”. Cei 40 de biți sunt structurați în cinci octeți: primii doi reprezintă valoarea umidității relative, următorii doi reprezintă temperatura, iar ultimul octet este suma de verificare, utilizată pentru validarea integrității transmisiei[13].

Alegerea senzorului DHT22 față de alternativa mai accesibilă, DHT11, a fost motivată de specificațiile tehnice superioare. DHT22 oferă o rezoluție de 0,1 $^{\circ}$ C pentru temperatură și de 0,1% pentru umiditate, față de 1 $^{\circ}$ C și, respectiv, 1% ale variantei DHT11. De asemenea, intervalul de măsurare al DHT22 este semnificativ mai larg: –40 $^{\circ}$ C până la +80 $^{\circ}$ C pentru temperatură și 0–100% pentru umiditate relativă, față de 0–50 $^{\circ}$ C și 20–80% ale DHT11. Aceste caracteristici îl recomandă pentru aplicații în care precizia și fiabilitatea pe termen lung sunt priorități, cum este cazul sistemului de monitorizare a microclimatului vegetal propus în cadrul acestei lucrări[13].

Din punct de vedere al conexiunilor fizice realizate pe breadboard, senzorul DHT22 a fost integrat în montajul experimental prin conectarea pinului VCC la bara de alimentare comună de 3,3 V, a pinului GND la bara de masă comună, iar pinul de date a fost conectat la pinul GPIO23 al microcontrolerului ESP32. Alegerea tensiunii de alimentare de 3,3 V asigură compatibilitatea directă cu nivelurile logice ale ESP32, eliminând necesitatea unui convertor suplimentar de nivel logic.

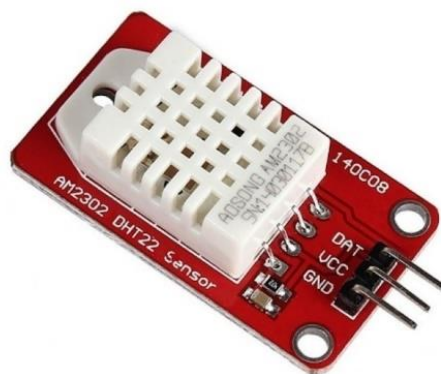


Figura 3.4. Senzorul DHT22

3.2.3. Senzor capacitiv de umiditate a solului

Spre deosebire de senzorii rezistivi clasici, care sunt predispuși la coroziune rapidă în sol, sistemul utilizează un senzor capacitiv. Acesta operează la tensiuni de 3,3 V – 5,5 V și generează un semnal analogic de ieșire direct proporțional cu conținutul volumetric de apă din solul plantei. Datele sunt citite prin portul ADC (Analog-to-Digital Converter) al modulului ESP32 pentru a depista prompt perioadele de stres hidric.

Principiul fizic pe care se bazează funcționarea senzorului capacitiv de umiditate a solului este variația constantei dielectrice a mediului dintre electrozi. Cei doi electrozi conductivi paraleli, împreună cu substratul de sol din care sunt separați printr-un strat izolator din PCB, formează un condensator al cărui parametru determinant este constanta dielectrică relativă a mediului înconjurător. Apa prezintă o constantă dielectrică foarte ridicată ($\epsilon \approx 80$), semnificativ mai mare decât cea a aerului ($\epsilon \approx 1$) sau a solului uscat ($\epsilon \approx 3-5$). Pe măsură ce conținutul volumetric de apă din substrat crește, constanta dielectrică efectivă a mediului dintre electrozi crește proporțional, determinând o creștere a capacității condensatorului format. Circuitul integrat intern al senzorului convertește această variație capacitivă într-un semnal analogic de tensiune, transmis pe pinul de ieșire AOUT[13].

Procesul de calibrare a senzorului s-a realizat prin măsurarea valorilor brute în două puncte de referință extreme: senzorul suspendat în aer uscat, caz în care portul ADC al ESP32 a returnat valoarea maximă de aproximativ 4095, și senzorul cufundat complet în apă, caz în care valoarea înregistrată a coborât la aproximativ 1500. Aceste două valori de referință stau la baza ecuației de mapare liniară implementată în firmware, prezentată în ecuația 5.1, prin care valoarea analogică brută este convertită într-un procentaj intuitiv de umiditate, cuprins între 0% și 100%.

3.2. Componente hardware

Față de senzorii rezistivi clasici, care măsoară conductivitatea electrică a solului prin trecerea unui curent între doi electrozi metalici expuși direct substratului, senzorul capacitiv prezintă avantajul major al longevității. Deoarece electrozii nu intră în contact direct cu soluția din sol, fenomenul de electroliză și coroziunea galvanică sunt practic eliminate, asigurând o durată de viață semnificativ mai mare în condiții de utilizare continuă.

Din punct de vedere al conexiunilor fizice realizate pe breadboard, pinul VCC al senzorului a fost conectat la bara de alimentare comună de 3,3 V, pinul GND la bara de masă comună, iar pinul de ieșire analogică AOUT a fost conectat la pinul GPIO35 al microcontrolerului ESP32, corespunzător canalului ADC15, compatibil cu convertorul analog-digital intern al acestuia.



Figura 3.5. Senzor capacitiv de umiditate a solului

3.2.4. Senzor de luminozitate BH1750

Senzorul BH1750 este un modul digital capabil să măsoare direct intensitatea luminoasă a mediului, returnând valorile în unitatea de măsură lux. Comunicarea cu ESP32 se realizează prin protocolul I2C, asigurând citiri precise indiferent de zgomotul de fond [14].

Principiul de funcționare al senzorului BH1750 se bazează pe efectul fotoelectric. O fotodiodă internă generează un curent proporțional cu intensitatea luminoasă incidentă, care este ulterior convertit într-o valoare digitală pe 16 biți prin intermediul unui convertor analog-digital integrat în circuitul senzorului. Rezultatul este transmis direct în unitatea standard de iluminare în lux, eliminând necesitatea oricărei conversii sau calibrări suplimentare la nivelul firmware-ului. Senzorul oferă trei moduri de rezoluție configurabile prin comandă software: modul de înaltă rezoluție continuă (0,5 lx/bit), modul de rezoluție standard (1 lx/bit) și modul de rezoluție redusă (4 lx/bit), cu intervale de măsurare cuprinse între 1 lx și 65.535 lx[14].

Comunicația cu microcontrolerul ESP32 se realizează prin protocolul I2C, un protocol serial sincron cu două fire: linia de date bidirecțională SDA și linia de ceas SCL. Senzorul BH1750 suportă două adrese I2C distincte, selectabile hardware prin pinul ADDR: adresa 0x23 când pinul ADDR este conectat la GND și adresa 0x5C când este conectat la VCC. Această facilitate permite conectarea simultană a două module BH1750 pe același bus I2C, fără conflicte de adresare. În cadrul proiectului de față a fost utilizată adresa implicită 0x23. Alegerea

protocolului I2C față de alternativa SPI este justificată de economia de pini GPIO: I2C necesită doar două fire de semnal indiferent de numărul de dispozitive conectate pe magistrală, față de minimum patru fire în cazul SPI, aspect important în contextul unui sistem cu mai mulți senzori conectați simultan la același microcontroler.

Din punct de vedere al conexiunilor fizice realizate pe breadboard, pinul VCC al senzorului a fost conectat la bara de alimentare comună de 3,3 V, pinul GND la bara de masă comună, pinul SDA la GPIO22, iar pinul SCL la GPIO21 al microcontrolerului ESP32, aceștia reprezentând pinii hardware dedicați magistralei I2C a cipului.



Figura 3.6. Senzorul de luminozitate BH1750

3.2.5. Montajul experimental pe breadboard

Pentru validarea designului, componentele au fost asamblate fizic pe un breadboard. Conexiunile logice au fost realizate între pinii senzorilor și pinii GPIO ai microcontrolerului ESP32. Sensorii cu ieșiri analogice au fost conectați la pinii compatibili cu multiplexorul ADC, iar cei digitali au fost conectați la pinii compatibili cu protocoalele I2C și One-Wire. Montajul centralizează atât cablurile de alimentare (3,3 V / 5 V și GND) cât și cablurile de date într-o machetă de testare flexibilă, permisivă față de modificările premergătoare instalării pe un circuit imprimat.

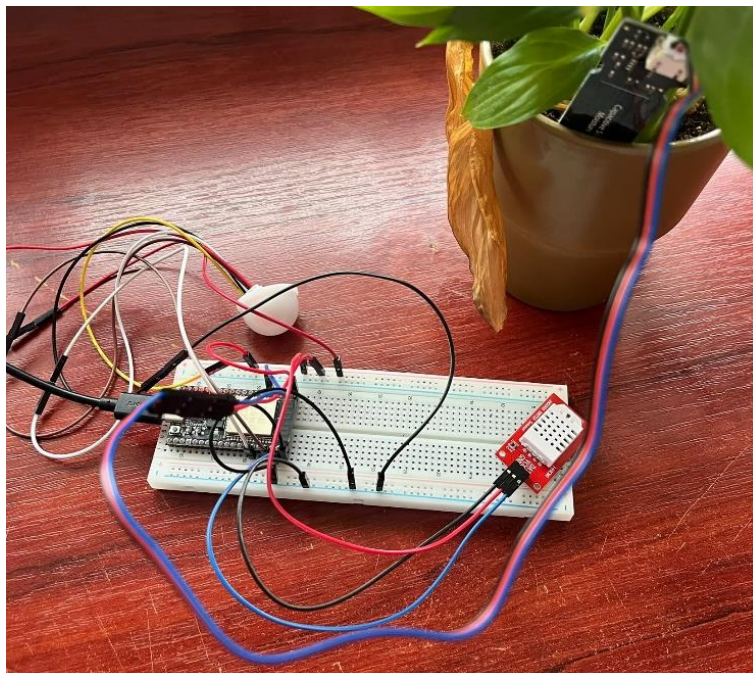


Figura 3.7. Montajul experimental al sistemului pe breadboard

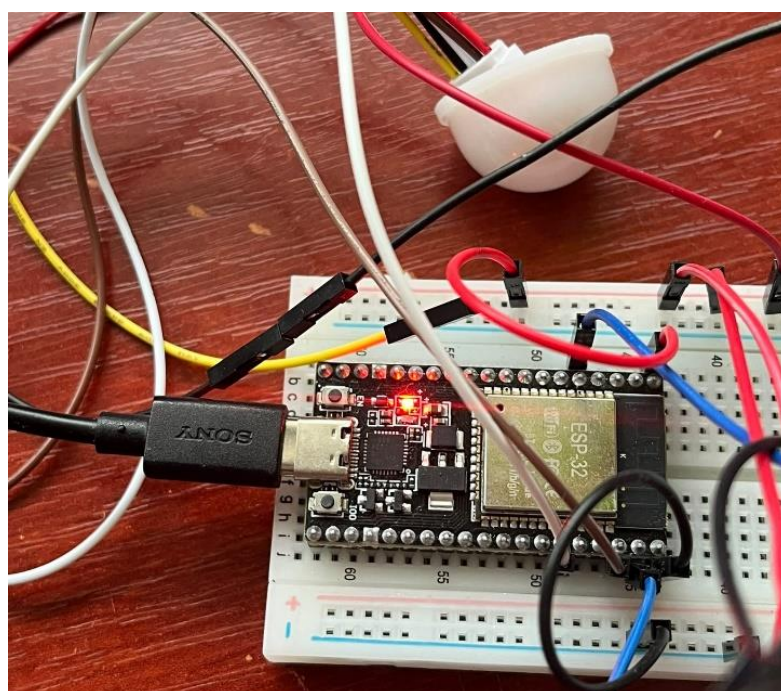


Figura 3.8. Montajul experimental al sistemului pe breadboard – ESP32 cu conexiunile senzorilor



Figura 3.9. Montajul experimental al sistemului pe breadboard împreună cu planta în suport

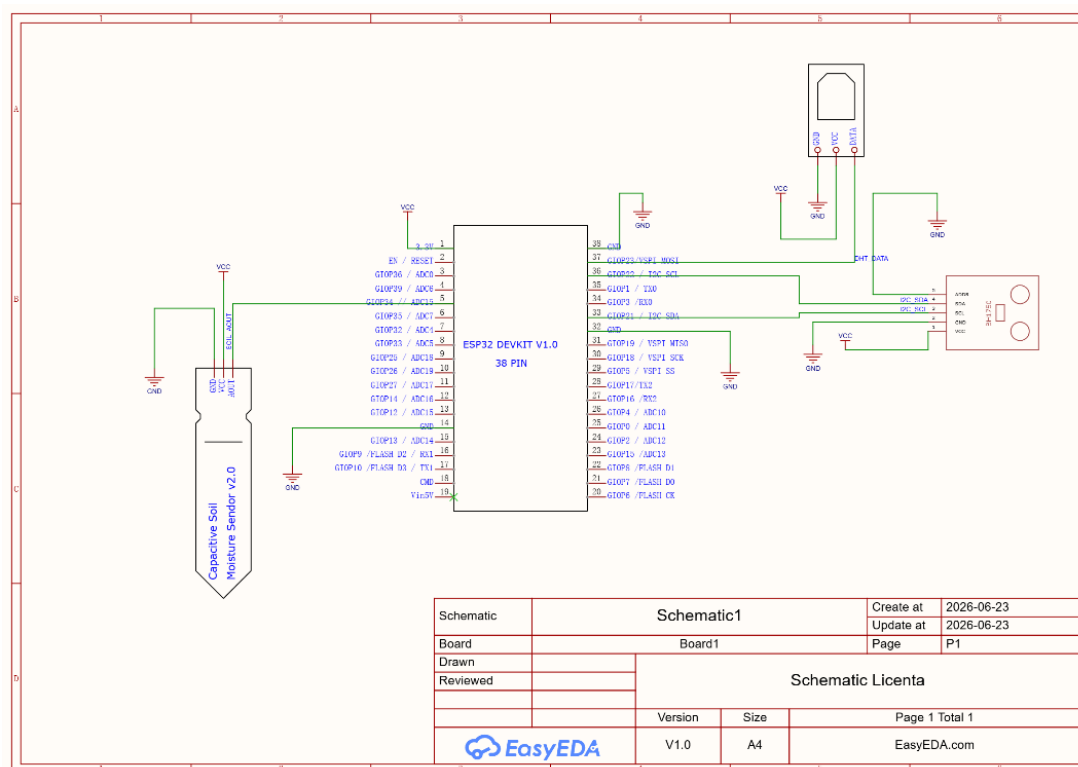


Figura 3.10. Schema electrică de interconectare a senzorilor de achiziție cu ESP32 în mediul EasyEDA

3.3. Software și protocoale de comunicație

3.3.1. Programarea ESP32(Arduino IDE)

Codul microcontrolerului a fost dezvoltat în limbajul C/C++ prin intermediul mediului Arduino IDE. Structura programului inițializează pinii de intrare și comunicația serială, citește periodic informațiile provenite de la senzori și se conectează la rețeaua Wi-Fi locală. Scriptul

3.3. Software și protocoale de comunicație

este conceput să împacheteze datele colectate sub forma unui șir de caractere și să le transfere asincron către server folosind o conexiune client-broker MQTT.

```
const char* ssid = "ESP32Test";
const char* password = "12345678";
const char* mqtt_server = "172.20.10.2";
const int mqtt_port = 1883;

WiFiClient espClient;
PubSubClient client(espClient);

#define DHTPIN 23
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);
#define SOIL_PIN 34
BH1750 lightMeter;
```

Figura 3.11. Includerea și definirea pinilor și a MQTT din firmware-ul ESP32

```
void setup_wifi() {
  Serial.println();
  Serial.print("Conectare la WiFi: ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi conectat!");
  Serial.print("Adresa IP ESP32: ");
  Serial.println(WiFi.localIP());
}
```

Figura 3.12. Funcția setup() pentru conectarea Wi-Fi și MQTT

```
char buffer[10];
dtostrf(temperaturaAer, 4, 1, buffer);
client.publish("smartgarden/temperatura", buffer);
dtostrf(umiditateAer, 4, 1, buffer);
client.publish("smartgarden/umiditate_aer", buffer);
dtostrf(luminaLux, 6, 1, buffer);
client.publish("smartgarden/lumina", buffer);
itoa(valoareSolBruta, buffer, 10);
client.publish("smartgarden/sol", buffer);
```

Figura 3.13. Publicarea datelor senzorilor pe topicurile MQTT din firmware-ul ESP32

3.3.2. Protocolul MQTT și brokerul Mosquitto

Transmisia datelor se bazează pe protocolul MQTT (Message Queuing Telemetry Transport). Acest protocol funcționează pe modelul Publisher-Subscriber și este special

conceput pentru dispozitive IoT cu resurse limitate, care operează la lățimi de bandă reduse [15]. Comparativ cu protocoalele alternative, precum HTTP sau CoAP, MQTT se distinge prin overhead minim: headerul are dimensiunea minimă de doar 2 bytes și prin latență redusă în condiții de rețea instabilă [15]. Microcontrolerul ESP32 acționează ca Publisher, publicând datele de la senzori pe topicuri prestabilite. În centrul acestei rețele se află brokerul Mosquitto, responsabil de preluarea și distribuirea mesajelor către orice client abonat, precum Node-RED. Această metodă non-blocantă garantează că placa de dezvoltare nu se blochează în timpul execuției din cauza latențelor de rețea [15].

Structura unui pachet MQTT este compusă din trei componente distincte: headerul fix, headerul variabil și payload-ul. Headerul fix, prezent în orice pachet MQTT, ocupă minimum 2 octeți și conține tipul mesajului (CONNECT, PUBLISH, SUBSCRIBE) și lungimea totală a pachetului codificată printr-un algoritm de lungime variabilă. Headerul variabil este opțional și conține informații specifice tipului de pachet, precum identificatorul mesajului sau numele topicului de publicare. Payload-ul reprezintă conținutul util efectiv al mesajului, adică datele transmise de la senzori către broker. Această arhitectură minimalistă conferă protocolului MQTT un overhead semnificativ mai redus față de HTTP, unde fiecare cerere implică transmiterea unor headere text de sute de octeți, și față de CoAP, care, deși compact, nu oferă același nivel de suport nativ pentru modelul publisher-subscriber în rețele locale.

În cadrul proiectului de față, topicurile MQTT au fost organizate ierarhic, urmând convenția standard de denumire, cu separatorul „/”, conform structurii prezentate în Tabelul 3.2.

Tabelul 3.2 Structura topicurilor MQTT utilizate în sistem

Topic	Senzor sursă	Conținut payload
Umiditate	DHT22	Valoare umiditate relativă aer în %
Temperatură	DHT22	Valoare temperatură în °C
Luminozitate	BH1750	Valoare brută ADC (0-4095)
Umiditate sol	Senzor capacitiv	Valoare intensitate luminoasă în lux

Microcontrolerul ESP32 acționează exclusiv ca Publisher, publicând periodic câte un mesaj pe fiecare topic. Brokerul Mosquitto, instalat local pe calculatorul gazdă, primește aceste mesaje și le redistribuie către toți clienții abonați. Platforma Node-RED se abonează simultan

3.3. Software și protocoale de comunicație

la toate cele patru topicuri prin noduri dedicate de tip mqtt-in, procesând fiecare mesaj la sosire, fără a bloca execuția fluxului pentru celelalte canale de date.

```
Wireless LAN adapter Wi-Fi:
Connection-specific DNS Suffix . : 
Link-local IPv6 Address . . . . . : fe80::7bf6:3e50:ff7b:e702%14
IPv4 Address. . . . . : 172.20.10.2
Subnet Mask . . . . . : 255.255.255.240
Default Gateway . . . . . : 172.20.10.1

Ethernet adapter Bluetooth Network Connection:
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
```

Figura 3.14. Adresa IP a calculatorului gazdă, rezultatul comenzii ipconfig

```
Conectare la broker MQTT...conectat!
AER -> Temp: 27.6°C | Umiditate: 69.5% || SOL -> Brut: 2885 || LUMINĂ -> 147.5 lux
AER -> Temp: 27.6°C | Umiditate: 69.6% || SOL -> Brut: 2885 || LUMINĂ -> 147.5 lux
AER -> Temp: 27.5°C | Umiditate: 69.7% || SOL -> Brut: 2896 || LUMINĂ -> 147.5 lux
AER -> Temp: 27.5°C | Umiditate: 69.7% || SOL -> Brut: 2896 || LUMINĂ -> 147.5 lux
AER -> Temp: 27.5°C | Umiditate: 69.7% || SOL -> Brut: 2880 || LUMINĂ -> 147.5 lux
AER -> Temp: 27.5°C | Umiditate: 69.8% || SOL -> Brut: 2906 || LUMINĂ -> 147.5 lux
AER -> Temp: 27.5°C | Umiditate: 69.8% || SOL -> Brut: 2890 || LUMINĂ -> 147.5 lux
```

Figura 3.15. Monitorizarea în timp real a parametrilor de mediu prin conexiunea MQTT

3.3.3. Platforma Node-RED

Node-RED deservește rolul de server de aplicație și de interfață grafică pentru utilizatorul final. Platforma primește în timp real telemetria de la senzorii publicați prin MQTT și o procesează cu ajutorul unui algoritm intern. Pe lângă afișarea parametrilor sub formă de grafice, dashboard-ul include o plantă animată cu expresii adaptive și un asistent virtual care evaluează datele primite, inclusiv rezultatele prelucrării imaginii trimise prin HTTP de la MATLAB și emite recomandări contextuale de îngrijire direct către utilizator.

Din punct de vedere arhitectural, flow-ul Node-RED implementat în cadrul proiectului este structurat în trei zone funcționale distincte, vizibile în Figura 3.16.

Prima zonă gestionează recepția și stocarea datelor de la senzori. Patru noduri de tip mqtt-in sunt configurate individual, fiecare abonând la câte unul dintre topicurile definite în Tabelul 3.2. La sosirea unui mesaj, payload-ul brut este extras și pasat unui nod de tip function, care identifică sursa prin câmpul msg.topic și aplică conversia necesară pentru senzorul de sol, conform ecuației 5.1. și stochează valoarea rezultată în contextul global al fluxului prin funcția *flow.set()*. Această abordare asigură persistența valorilor între mesajele individuale, permițând algoritmului de decizie să acceseze întotdeauna cel mai recent eșantion valid pentru fiecare parametru, indiferent de ordinea sosirii mesajelor pe topicuri diferite.

A doua zonă gestionează recepția asincronă a rezultatelor analizei vizuale transmise din mediul MATLAB. Un nod de tip http-in deschide un endpoint dedicat pe portul 1880, ascultând cererile HTTP POST primite de la scriptul de procesare a imaginilor. La recepție, payload-ul

JSON este parsat automat, iar procentele cromatice extrase sunt stocate în contextul fluxului sub chei dedicate, alături de datele senzorilor fizici.

A treia zonă cuprinde logica de decizie și actualizarea interfeței grafice. Un nod central de tip funcție interoghează periodic toate valorile stocate în contextul fluxului, aplică algoritmul de decizie ierarhic descris în Capitolul 5 și generează mesajul de stare și recomandarea textuală corespunzătoare. Rezultatele sunt transmise simultan către nodurile de tip *ui_gauge*, *ui_chart* și *ui_template*, care actualizează în timp real widgeturile corespunzătoare din dashboard prin conexiunea WebSocket menținută activă între serverul Node-RED și browserul clientului.

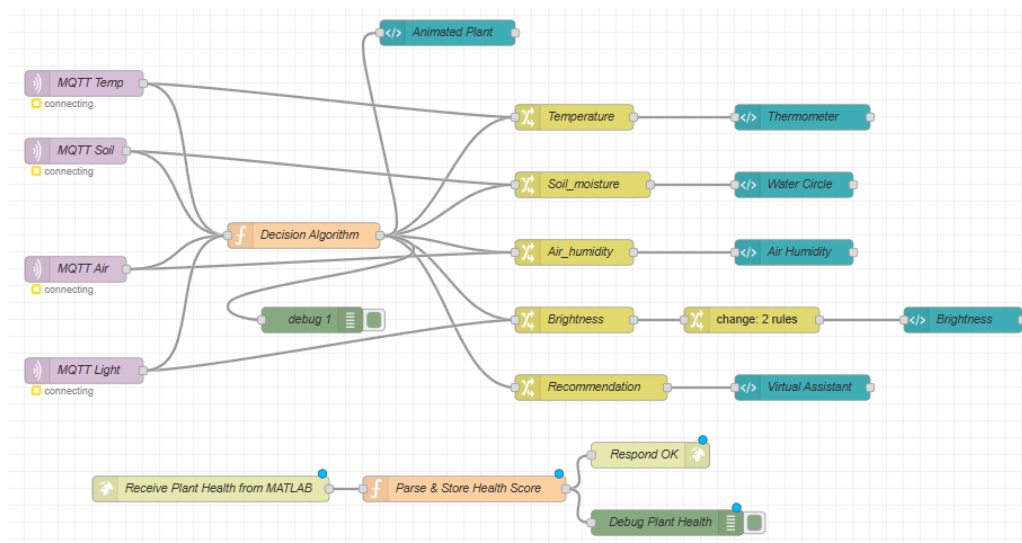


Figura 3.16. Flow-ul Node-RED al sistemului

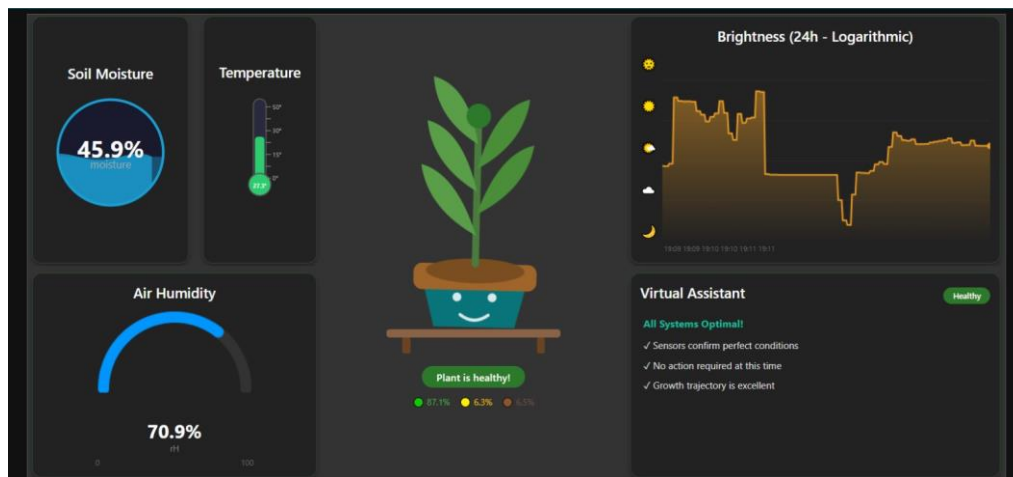


Figura 3.17. Dashboard-ul interactiv Node-RED

3.3. Software și protocoale de comunicație

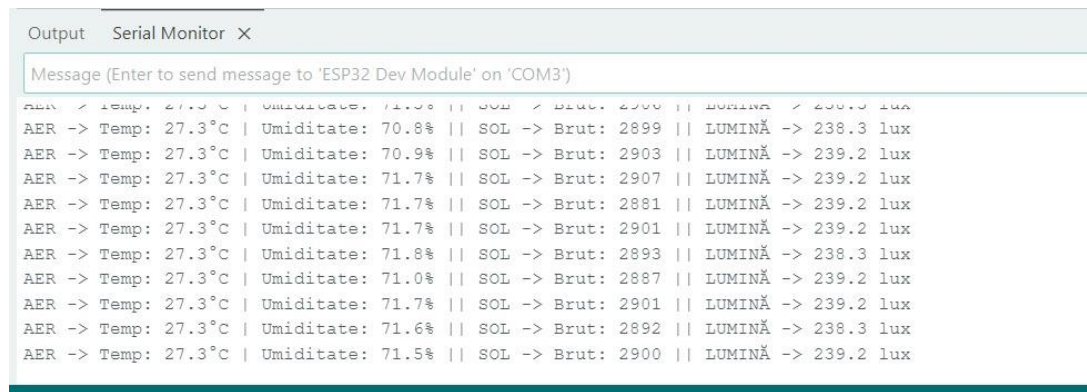


Figura 3.18. Serial Monitor cu valorile trimise ulterior spre Node-RED

4. Analiza stării plantei prin procesare de imagini(MATLAB)

Achiziția parametrilor fizici ai microclimatului (temperatura, umiditatea aerului, umiditatea solului și iluminarea) oferă o imagine clară asupra condițiilor de mediu în care se dezvoltă planta, însă nu poate reflecta direct starea fiziologică curentă a țesutului vegetal. Din acest motiv, pentru a construi un sistem complet și fiabil de monitorizare, arhitectura hibridă propusă integrează un subsistem complementar de viziune artificială implementat în mediul MATLAB.

Rolul acestui capitol este de a detalia algoritmul de procesare numerică proiectat pentru izolarea automată a aparatului foliar de elementele de fundal și pentru clasificarea cromatică a pixelilor. Prin această abordare, datele fizice discrete culese de senzori sunt corelate direct cu indicatori vizuali cantitativi. În paginile următoare sunt prezentate metodologia de segmentare spectrală bazată pe clustere, ecuațiile matematice de conversie și pragurile de decizie implementate, precum și mecanismul de interconectare asincronă prin rețea cu platforma centrală de dispecerizare.

4.1. Metoda de segmentare *K-means* în spațiul RGB

Subsistemul de analiză vizuală utilizează tehnici de procesare numerică a imaginilor pentru a izola aparatul foliar și pentru a diagnostica starea de sănătate a acestuia. În prima etapă, imaginea brută achiziționată în format RGB este supusă unui proces de filtrare a fundalului (reducerea zgomotului optic provocat de pereți, reflexii sau ghiveci). Deoarece spațiul RGB este sensibil la variațiile de iluminare directă, izolarea se realizează prin conversia matricei de pixeli în spațiul HSV (Hue, Saturation, Value). Se generează o mască binară bazată pe praguri stricte de saturație și strălucire, conform ecuației:

$$P_{mask} = (S \geq 0.30) \cap (V \geq 0.10) \quad (4.1)$$

Procesarea digitală a imaginilor în contextul diagnosticării biomasei vegetale implică izolarea prealabilă a canalelor de culoare capabile să ofere cel mai ridicat grad de contrast între țesutul foliar și elementele adiacente din scenă. Deși camerele digitale achiziționează în mod nativ informația sub forma unei matrice tridimensionale RGB, acest spațiu cromatic prezintă limitări severe atunci când este utilizat direct în algoritmi de segmentare automată în condiții reale. Problema fundamentală rezidă în corelația strânsă dintre cele trei componente (roșu,

4.1. Metoda de segmentare K-means în spațiul RGB

verde și albastru) și intensitatea luminii ambientale. O schimbare minoră a iluminării din încăpere, provocată de nori sau de modificarea poziției soarelui, alterează simultan valorile numerice ale tuturor celor trei canale pentru același pixel foliar, determinând eșecul algoritmilor bazate pe praguri fixe.

Pentru a contracara această sensibilitate, s-a implementat o trecere intermediară a matricei de pixeli în spațiul HSV. Acest model decuplează nuanța cromatică pură de factorii de strălucire și de umbră. Componenta de saturație indică puritatea culorii, permițând distingerea frunzelor vii de suprafețele mate ale pereților din fundal, în timp ce valoarea luminozității permite filtrarea zonelor profund întunecate sau a umbrelor proiectate în spatele ghiveciului. Prin aplicarea mascată a acestor filtre binare, algoritmul K-means primește ca date de intrare un set de vectori curățați de zgomot optic, ceea ce duce la o reducere semnificativă a numărului de iterații necesare pentru atingerea convergenței și previne deplasarea eronată a centroizilor către culori străine ale biomasăi reale a plantei analizate.

Pixelii care nu îndeplinesc simultan aceste condiții sunt eliminați, optimizând timpul de calcul prin reținerea exclusivă a coordonatelor cromatice ale plantei în vectorii de analiză. Pentru segmentarea propriu-zisă a țesuturilor se implementează algoritmul K-means, construit în mod nativ în limbajul MATLAB, fără apelarea unor funcții predefinite din biblioteci externe (Toolboxes), aspect care evidențiază caracterul original al implementării. Algoritmul este configurat pentru un număr de clustere $NC = 6$. Această rezoluție cromatică fină este critică: alegerea unui număr mai mic de clase ar duce la absorbția zonelor mici afectate (cum sunt vârfurile uscate sau petele incipiente de boală) în grupurile majore de verde sănătos. Pentru a asigura o convergență rapidă și a evita selectarea unor centroizi identici, algoritmul folosește o metodă originală de preprocesare: vectorii tridimensionali RGB ai pixelilor reținuți sunt compactați într-o reprezentare scalară unică pe 24 de biți prin transformarea:

$$V = R \cdot 256^2 + G \cdot 256 + B \quad (4.2)$$

Acest vector este sortat, iar prin calcularea diferențelor adiacente se elimină toate duplicatele cromatice. Inițializarea celor NC centroizi se realizează prin selecție aleatorie strict din această listă de culori unice, utilizând o permutare pseudoaleatoare cu seed fixat grafic pentru a garanta reproductibilitatea perfectă a testelor. Ulterior, algoritmul rulează iterativ o buclă de optimizare. În cadrul fiecărei iterații, se calculează distanța euclidiană în spațiul tridimensional RGB de la fiecare pixel al plantei la fiecare centroid, conform formulei:

$$d = \sqrt{(R_i - R_k)^2 + (G_i - G_k)^2 + (B_i - B_k)^2} \quad (4.3)$$

Pixelii sunt alocați instantaneu clusterului cu cea mai mică distanță. Ulterior, poziția fiecărui centroid este recalculată ca medie aritmetică a componentelor cromatice ale tuturor pixelilor arondați la clasa respectivă. Procesul se repetă iterativ până la stabilizarea completă a centroizilor, definită matematic prin scăderea sumei absolute a diferențelor de poziție sub pragul numărului de clustere, sau la atingerea unui plafon de siguranță de 50 de iterații:

$$\sum |c_{new} - c_{old}| < NC \quad (4.4)$$

Alegerea numărului de clustere, $NC = 6$, reprezintă un compromis optim între granularitatea cromatică necesară diagnosticării foliare și costul computațional al algoritmului. O valoare mai mică, precum $NC = 3$, ar fi insuficientă pentru a captura nuanțele intermediare ale țesutului vegetal: zonele de tranziție dintre verde sănătos și galben stresat, care apar frecvent în stadiile incipiente ale clorozei, ar fi absorbite fie în clusterul verde dominant, fie în cel galben, mascând astfel degradarea timpurie. La polul opus, un număr excesiv de clustere, precum $NC = 10$ sau mai mult, ar fragmenta excesiv suprafața foliară în nuanțe cromatice atât de apropiate încât clasificarea semantică ulterioară în categoriile verde, galben și maro ar deveni ambiguă, crescând totodată semnificativ numărul de iterații necesare pentru atingerea convergenței și, implicit, timpul de execuție al scriptului.

La fiecare iterație, atât etapa de alocare a pixelilor la cel mai apropiat centroid, cât și etapa de recalculare a centroizilor ca medie aritmetică a punctelor alocate reduc sau mențin constantă valoarea acestei funcții. Deoarece numărul de configurații posibile de alocare este finit, algoritmul este garantat să convergă într-un număr finit de iterații. Plafonul de siguranță de 50 de iterații implementat în scriptul MATLAB reprezintă o măsură de protecție împotriva cazurilor patologice în care doi centroizi oscilează între două configurații echivalente energetice, prevenind astfel blocarea execuției scriptului în bucle infinite.



Figura 4.1. Imaginea originală a plantei supusă analizei vizuale

Odată ce imaginea originală este încărcată în bufferul grafic și afișată în cadrul interfeței (conform figurii 4.1), sistemul inițiază faza latentă de preprocesare spectrală. Eliminarea pixelilor de fundal prin aplicarea măștii binare reduce volumul de date, păstrând în matricea de calcul exclusiv regiunile care prezintă caracteristici specifice biomasei active. Ulterior, prin rularea iterativă a algoritmului K-means pe parcursul mai multor cicluri de optimizare, se obține convergența către centrozii cromatici. Acest proces rearanjează întreaga structură de pixeli a frunzei în funcție de nuanțele sale dominante. Rezultatul final al acestei segmentări geometrice, în care fundalul este complet izolat și reprezentat prin valori nule (negru pur), iar planta este clar divizată în cele $NC = 6$ zone cromatice distincte, este ilustrat în detaliu în Fig. 4.2.

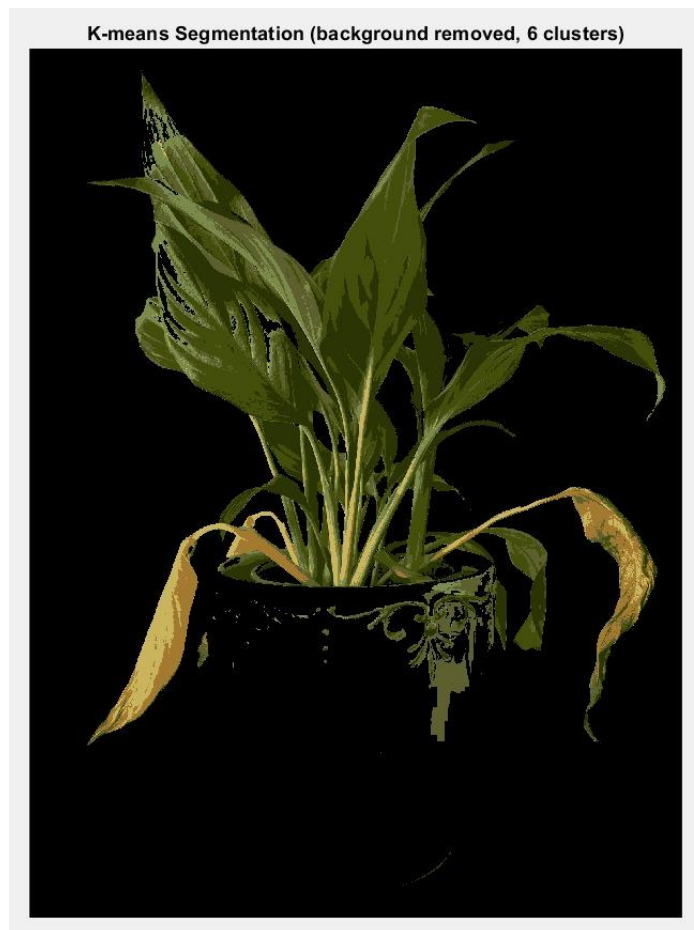


Figura 4.2. Rezultatul segmentării prin algoritmul K-means cu 6 clustere (fundal eliminat din mască)

4.2. *Calculul procentajelor de sănătate a plantei*

După convergența algoritmului și obținerea celor 6 nuanțe dominante ale centrozilor, este necesară clasificarea semantică a acestora în categorii medicale-vegetale. Deoarece interpretarea nuanțelor este dificilă în coordonatele brute RGB, centrozii finali sunt convertiți în spațiul HSV, extrăgându-se componenta Hue (nuanța spectrală, exprimată ca unghi pe cerul culorilor). Clasificarea deterministă a fiecărui cluster se realizează automat la nivel software prin aplicarea următoarelor reguli stricte de prag:

- $H \geq 0.16 \rightarrow \text{\textit{\text{Țesut sănătos (Verde)}}$
- $0.13 \leq H < 0.16 \rightarrow \text{\textit{\text{Țesut stresat fiziologic (Galben)}}$
- $0.02 \leq H < 0.13 \rightarrow \text{\textit{\text{Țesut necrozat/uscăat (Maro)}}$

Orice altă valoare care pică în afara acestor intervale vegetale primește eticheta „other” și este exclusă din calculul patologic, asigurând eliminarea oricăror artefacte geometrice rămase. Prin însumarea pixelilor aparținând exclusiv clusterelor foliare valide și raportarea lor la masa totală de pixeli ai plantei, se calculează ponderile procentuale finale:

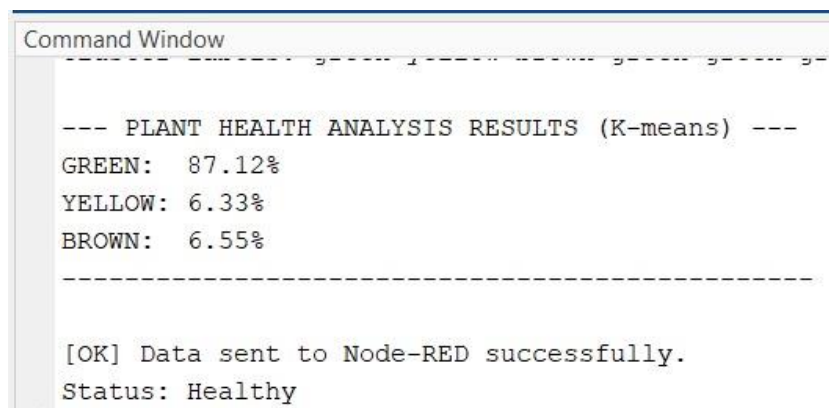
$$\% \text{\textit{\text{Țesut}}} = \frac{\sum \text{Pixeli_Cluster_Categorie}}{\text{Total_Pixeli_Planta}} \times 100 \quad (4.4)$$

4.2. Calculul procentajelor de sănătate a plantei

Pragurile valorii Hue utilizate pentru clasificarea semantică a clusterelor au fost stabilite pe baza proprietăților spațiului cilindric HSV și a caracteristicilor spectrale ale pigmentilor vegetali. În modelul HSV, componenta Hue este exprimată ca un unghi normalizat în intervalul $[0, 1]$, corespunzând unui cerc cromatic complet de 360° . Pigmentul clorofilă, responsabil de culoarea verde a țesutului foliar sănătos, absoarbe preponderent lungimile de undă din spectrul roșu și albastru, reflectând lumina verde, ceea ce plasează centroizii clusterelor de țesut sănătos în intervalul Hue $H \geq 0,16$, corespunzător unghiurilor de peste 57° pe cercul cromatic. Degradarea clorofilei și acumularea de pigmenți, caracteristice stresului fiziologic, deplasează culoarea frunzei către galben, în intervalul corespunzător valorilor $0,13 \leq H < 0,16$. Necroza și deshidratarea severă a țesutului, asociate pigmentilor bruni, sunt captate în intervalul $0,02 \leq H < 0,13$.

Este important de menționat că aceste praguri au fost validate experimental pe specimene vegetale testate în cadrul proiectului și pot necesita ajustări minore pentru specii cu pigmentație atipică, precum plantele cu frunze roșii sau purpurii, la care distribuția Hue diferă fundamental față de modelul foliar verde clasic. Această limitare este recunoscută și constituie una dintre direcțiile de dezvoltare viitoare identificate în Capitolul 8, unde se propune înlocuirea regulilor deterministe de prag cu un model de clasificare antrenat supervizat, capabil să generalizeze pe un spectru mai larg de specii vegetale.

Pe baza acestor ponderi se stabilește statusul global trimis utilizatorului. Dacă procentul de țesut verde depășește sau este egal cu pragul critic de 60%, starea generală este definită ca fiind excelentă („Healthy”). Coborârea sub acest prag, dar menținerea peste 35%, plasează sistemul în stare de avertizare („Warning”), în timp ce o pondere a țesutului verde mai mică de 35% indică o degradare foliară severă, clasificată ca „Critical”.

A screenshot of a MATLAB Command Window. The title bar says "Command Window". The text inside shows the results of a K-means analysis for plant health. It lists three categories: GREEN at 87.12%, YELLOW at 6.33%, and BROWN at 6.55%. Below this, a status message indicates that data was sent to Node-RED successfully and the plant status is "Healthy".

```
--- PLANT HEALTH ANALYSIS RESULTS (K-means) ---
GREEN:  87.12%
YELLOW:  6.33%
BROWN:  6.55%

-----

[OK] Data sent to Node-RED successfully.
Status: Healthy
```

Figura 4.3. Rezultatele analizei cromatice din MATLAB

4.3. Integrarea cu Node-RED prin HTTP

Ultima etapă a scriptului MATLAB o reprezintă exportul datelor către nivelul middleware-ului. Informațiile nu rămân izolate în mediul de simulare, ci sunt transmise asincron prin rețea utilizând protocolul HTTP (Hypertext Transfer Protocol).

Datele rezultate din analiză (ponderile exprimate în variabilele `percentGreen`, `percentYellow`, `percentBrown` și stringul de diagnosticare `status`) sunt încapsulate automat sub forma unui obiect JSON (JavaScript Object Notation) compact. Transmisia se realizează prin metoda de rețea HTTP POST, apelând funcția nativă `webwrite` către un endpoint dedicat, deschis pe serverul Node-RED, la adresa IP `http://172.20.10.2:1880/plant-health`.

La recepție, Node-RED parsează automat corpul solicitării, asociază procentele vizuale calculate în MATLAB cu valorile primite în timp real de la senzori prin protocoalele MQTT și actualizează interfața grafică utilizator, închizând astfel bucla completă de monitorizare hibridă.

Structura obiectului JSON transmis prin cererea HTTP POST către endpoint-ul Node-RED este compusă din patru câmpuri distincte, fiecare corespunzând unui indicator analitic calculat de algoritmul K-means.

Câmpurile `percentGreen`, `percentYellow` și `percentBrown` conțin valorile procentuale ale țesutului foliar clasificat în fiecare categorie, exprimate cu două zecimale, iar câmpul `status` conține șirul de caractere corespunzător stării globale, determinată pe baza pragurilor descrise în secțiunea anterioară. Această structură minimalistă a fost aleasă deliberat pentru a menține dimensiunea payload-ului la valori reduse, compatibile cu limitările de lățime de bandă ale rețelei locale, și pentru a facilita parsarea rapidă în nodul de tip `http-in` din Node-RED, prin funcția nativă `JSON.parse()`.

Pentru gestionarea situațiilor în care conexiunea de rețea dintre calculatorul gazdă și serverul Node-RED este temporar indisponibilă, funcția nativă `webwrite` din MATLAB include un mecanism de timeout configurabil. În cazul în care serverul nu răspunde în intervalul de timp alocat, scriptul generează un mesaj de eroare în linia de comandă MATLAB și continuă execuția fără a bloca analiza vizuală curentă. Această abordare asigură robustețea subsistemului de viziune artificială față de instabilitățile tranzitorii ale rețelei locale, garantând că o întrerupere temporară a conexiunii nu afectează integritatea calculelor cromatice efectuate local în mediul MATLAB.



Figura 4.4. Structura fluxului de recepție asincronă HTTP POST în Node-RED

5. Algoritmul de decizie și sistemul de recomandări

Unul dintre obiectivele principale ale sistemului este capacitatea de a lua decizii autonome cu privire la starea plantei și de a comunica utilizatorului acțiunile necesare într-un mod clar și intuitiv. Pentru a atinge acest obiectiv, a fost proiectat și implementat un algoritm de decizie care centralizează datele provenite din două surse complementare: senzorii fizici conectați la microcontrolerul ESP32 și analiza vizuală a imaginilor plantei realizată în MATLAB.

Abordarea aleasă combină monitorizarea continuă a parametrilor de mediu (temperatură, umiditate a aerului, umiditate a solului și luminozitate) cu o evaluare periodică a stării vizuale a frunzelor, pe baza distribuției culorilor detectate de algoritmi de procesare a imaginilor. Această combinație permite sistemului să detecteze atât probleme imediate, cum ar fi solul uscat sau temperatura excesivă, cât și degradări lente și progresive ale sănătății plantei, vizibile prin îngălbenirea sau brunificarea frunzelor înainte ca senzorii să indice o valoare critică.

Întreaga logică de decizie este implementată în mediul Node-RED sub forma unui nod de tip function, care primește în timp real datele de la senzori prin protocolul MQTT și rezultatele analizei vizuale prin cereri HTTP POST trimise din MATLAB. Pe baza acestor date, algoritmul determină una dintre cele trei stări posibile ale plantei sănătoase, în atenție sau în pericol și generează o recomandare textuală personalizată, afișată utilizatorului prin componenta Virtual Assistant din interfața grafică.

5.1. Praguri și criterii utilizate

Algoritmul de decizie reprezintă nucleul logic al sistemului Smart Garden și este implementat ca un nod de tip funcție în mediul Node-RED. Rolul său este de a centraliza toate datele provenite din surse multiple de senzori fizici și de analiză a imaginii și de a determina starea curentă a plantei precum și recomandarea corespunzătoare pentru utilizator.

Criteriile de evaluare sunt bazate pe patru parametri măsurabili în timp real:

- Temperatura aerului — măsurată în grade Celsius prin senzorul DHT22
- Umiditatea aerului — exprimată procentual, tot prin DHT22
- Umiditatea solului — exprimată procentual după conversia valorii brute a senzorului capacitiv
- Luminozitatea ambientală — exprimată în lux prin senzorul de lumină

Pragurile de decizie prezentate în Tabelul 5.1 au fost stabilite pe baza cerințelor biologice generale documentate în literatura de specialitate privind plantele de interior. Intervalul optim de umiditate a solului, cuprins între 45% și 100%, corespunde nivelului de saturație hidrică și transportului activ al nutrienților prin sistemul radicular. Coborârea umidității sub pragul de 25% indică un deficit hidric sever, în care presiunea scade sub nivelul critic, declanșând mecanisme de închidere a stomatelor și blocând procesul de fotosinteză. Intervalul de atenție cuprins între 25% și 45% semnalează o reducere progresivă a rezervelor hidrice, în care planta începe să manifeste primul stadiu de stres, fără a fi încă afectată ireversibil.

Pentru parametrul de temperatură, intervalul optim de 12 °C–30 °C corespunde ferestrei termice în care enzimele implicate în metabolismul vegetal funcționează la eficiență maximă. Depășirea pragului superior de 33 °C accelerează rata de transpirație foliară și poate provoca denaturarea proteinelor enzimatică, în timp ce coborârea sub 12 °C încetinește dramatic procesele metabolice și poate induce leziuni ale membranelor celulare prin cristalizarea apei intracelulare. În ceea ce privește luminozitatea, pragul de atenție stabilit la 800 de lux delimitează zona de iluminare excesivă, în care radiația solară directă poate provoca fotooxidarea pigmentilor clorofilieni și arsuri foliare prin supraîncălzirea locală a țesutului.

Pragurile utilizate în luarea deciziilor au fost stabilite pe baza cerințelor biologice generale ale plantelor de interior și sunt prezentate în tabelul următor:

Tabelul 5.1 Pragurile de decizie utilizate pentru evaluarea parametrilor de mediu

Parametru	Prag critic (stare tristă)	Prag atenție (stare neutră)	Interval optim
Umiditate sol	< 25%	25% – 45%	> 45%
Temperatură	> 33°C sau < 12°C	> 33°C sau < 12°C	12°C – 30°C
Luminozitate	—	> 800 lux	< 800 lux

```
if (Soil_moisture < 25) {
  stare = "trista";
  culoare = "#8B6914";
  Recommendation = "Uda urgent! Solul este prea uscat.";
} else if (Temperature > 33) {
  stare = "trista";
  culoare = "#8B6914";
  Recommendation = "Canicula! Muta planta la racoare.";
} else if (stare_viz === "trista" && Soil_moisture < 50) {
  stare = "trista";
  culoare = "#8B6914";
  Recommendation = "Planta arata rau (" + vizual.maro + "% maro). Verifica si uda.";
} else if (Brightness > 800) {
  stare = "neutra";
  culoare = "#C8A000";
  Recommendation = "Prea mult soare! Muta intr-un loc umbros.";
} else if (Soil_moisture < 45) {
  stare = "neutra";
  culoare = "#C8A000";
  Recommendation = "Solul incepe sa se usuce. Uda in curand.";
} else if (stare_viz === "fericita" && Soil_moisture > 45 && Temperature < 30) {
  stare = "fericita";
  culoare = "#2D7A2D";
  Recommendation = "Planta arata sanatoasa! Senzori si camera confirma.";
} else if (Temperature < 12) {
  stare = "neutra";
  culoare = "#C8A000";
  Recommendation = "Temperatura prea scazuta. Muta planta la caldura.";
} else {
  stare = "fericita";
  culoare = "#2D7A2D";
  Recommendation = "Planta este sanatoasa! Continua ingrijirea normala.";
}
```

Figura 5.1. Condițiile algoritmului de decizie implementat în Node-RED

5.2. Logica de combinare a datelor din senzori și analiza de imagine

Algoritmul funcționează în două etape distincte: colectarea și stocarea datelor, urmate de evaluarea și luarea deciziei.

Etapa 1 - Colectarea datelor

Deoarece senzorii publică date pe topicuri MQTT separate și independente, nodul de decizie primește mesaje individuale pentru fiecare parametru. La fiecare mesaj primit, algoritmul identifică sursa prin câmpul *msg.topic* și stochează valoarea corespunzătoare în contextul flow-ului Node-RED folosind funcția *flow.set()*:

```

if (topic.includes("temperatura")) {
    flow.set("real_Temperature", parseFloat(msg.payload));
} else if (topic.includes("umiditate_aer")) {
    flow.set("real_Air_humidity", parseFloat(msg.payload));
} else if (topic.includes("sol")) {
    let brut = parseFloat(msg.payload);
    let procentSol = ((4095 - brut) / (4095 - 1500)) * 100;
    if (procentSol < 0) procentSol = 0;
    if (procentSol > 100) procentSol = 100;

    flow.set("real_Soil_moisture", procentSol);
} else if (topic.includes("lumina")) {
    flow.set("real_Brightness", parseFloat(msg.payload));
}

```

Figura 5.2. Recepția și stocarea datelor de la senzori prin topicuri MQTT

Un aspect important este conversia valorii brute a senzorului de umiditate a solului. Senzorul returnează o valoare analogică între 1500 (sol complet ud) și 4095 (aer uscat), care este transformată într-un procentaj intuitiv de 0–100% prin formula:

$$\%Sol = \frac{4095 - \text{valoare_bruta}}{4095 - 1500} \times 100 \quad (5.1)$$

Etapa 2 – Analiză vizuală și combinarea surselor

Pe lângă datele de la senzori, algoritmul integrează și rezultatele analizei de imagine efectuate în MATLAB, folosind algoritmul K-Means de clusterizare. Aceste rezultate, procentele de pixeli verzi, galbeni și maronii din imaginea plantei sunt stocate în contextul flow-ului sub cheia *real_vizual*, iar pe baza acestor procente, algoritmul determină o stare vizuală intermediară:

```

const vizual      = flow.get("real_vizual")      || { verde: 60, galben: 20, maro: 20 };

let stare, Recommendation, culoare;

let stare_viz;
if (vizual.verde > 55) {
    stare_viz = "fericita";
} else if (vizual.maro > 35) {
    stare_viz = "trista";
} else {
    stare_viz = "neutra";
}

```

Figura 5.3. Integrarea rezultatelor analizei vizuale din MATLAB în algoritmul de decizie

Această stare vizuală este folosită ulterior ca factor suplimentar în decizia finală, creând un sistem de confirmare dublă: atât senzorii, cât și camera trebuie să indice o problemă pentru ca alarma să fie declanșată cu prioritate maximă.

Mecanismul de confirmare dublă implementat în algoritmul de decizie reprezintă unul dintre elementele de originalitate ale arhitecturii propuse. Spre deosebire de sistemele clasice care bazează diagnosticul exclusiv pe datele senzorilor fizici sau exclusiv pe analiza vizuală, sistemul propus corelează obligatoriu ambele surse de informație înainte de a emite o alertă de prioritate maximă. Concret, o stare critică este declanșată cu severitate maximă doar atunci când

5.3. Generarea recomandărilor pentru utilizator

atât senzorii fizici, cât și analiza cromatică K-means confirmă simultan existența unei probleme. De exemplu, o valoare scăzută a umidității solului cu un procent ridicat de țesut maro detectat vizual, conduce la o alertă critică imediată, în timp ce o umiditate scăzută, însoțită de un procent de țesut verde de peste 60%, generează doar o avertizare de atenție, sugerând că planta nu a acumulat încă stres vizibil la nivel foliar și că irigarea preventivă este suficientă.

Pentru stabilizarea interfeței grafice în condițiile fluctuațiilor minore ale citirilor senzorilor la granița dintre praguri, algoritmul implementează un mecanism de filtrare prin histerezis. Starea anterioară a sistemului este stocată în contextul fluxului Node-RED, iar o tranziție către o nouă stare este validată doar dacă depășirea pragului este confirmată pe parcursul a minimum 3 eșantioane consecutive. Această barieră temporală de validare previne oscilațiile rapide ale avatarului grafic și ale textului asistentului virtual în scenarii în care, de exemplu, temperatura fluctuează repetat în jurul valorii de 33 °C din cauza curenților de aer sau a variațiilor minore ale senzorului DHT22. Pragul de trei eșantioane consecutive a fost ales ca un compromis între reactivitatea sistemului la evenimente reale și imunitatea sa la perturbațiile tranzitorii de rețea sau de citire hardware.

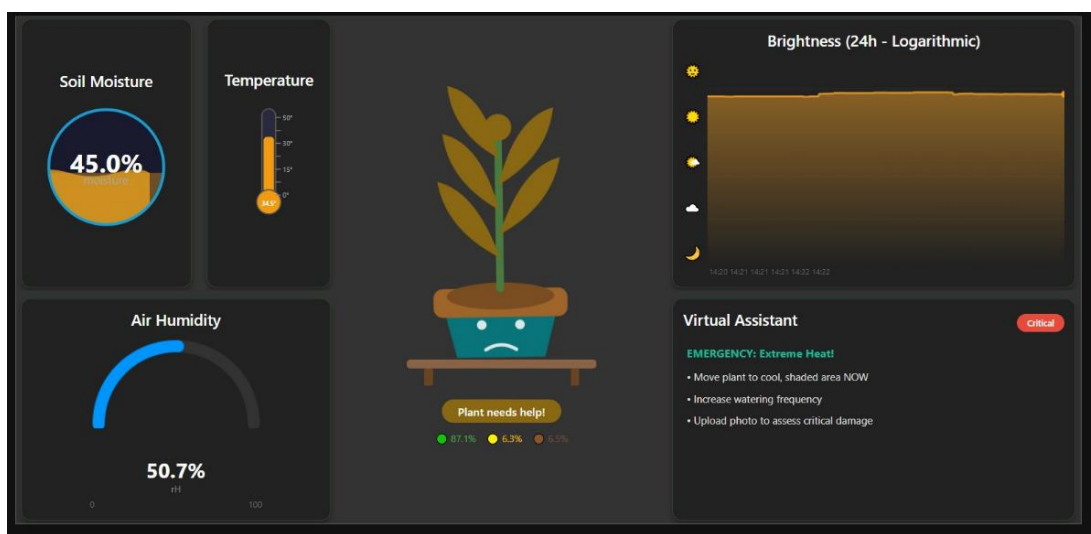


Figura 5.4. Interfața dashboard în stare critică

5.3. Generarea recomandărilor pentru utilizator

Decizia finală este luată printr-un lanț de condiții ordonate după prioritate, în care condițiile critice sunt verificate mai întâi. Această abordare garantează că situațiile de urgență sunt întotdeauna semnalate, indiferent de celelalte valori. Proiectarea ierarhică a structurii condiționale din cadrul nodului de decizie este esențială pentru prevenirea conflictelor logice atunci când sistemul hardware înregistrează abateri simultane de la stările optime ale microclimatului. În situații de stres ecologic compus, cum ar fi un scenariu în care solul prezintă un deficit hidric sever concomitent cu o expunere la o intensitate luminoasă dăunătoare,

algoritmul acordă prioritate absolută parametrului cu cel mai mare potențial de degradare biologică imediată a plantei. Întrucât deshidratarea la nivelul rădăcinii declanșează colapsul celular mult mai rapid decât expunerea termică tranzitorie, ramura de verificare a umidității din sol este evaluată prima în ordinea execuției instrucțiunilor din firmware-ul middleware-ului. Acest mod de sortare pe niveluri de severitate elimină riscul ca o alertă minoră să mascheze o urgență critică.

Un alt aspect fundamental implementat în logica de decizie vizează atenuarea zgomotului de comutație la granița dintre stările sistemului, fenomen cunoscut în ingineria sistemelor automate sub denumirea de filtrare prin histerezis. Deoarece citirile senzorilor fizici pot prezenta fluctuații minore la limita dintre pragurile prestabilite (de exemplu, o oscilație repetată a temperaturii în jurul valorii de 33 de grade Celsius), un algoritm rigid ar genera o schimbare extrem de rapidă și deranjantă a avatarului plantei și a textului asistentului virtual. Pentru a stabiliza interfața grafică, algoritmul stochează starea anterioară și impune o barieră temporală de validare. O nouă recomandare textuală este trimisă către widget-ul de afișare din dashboard doar dacă depășirea pragului critic se menține consecutiv pe parcursul mai multor eșantioane de date confirmate, asigurând astfel o diagnoză stabilă, imună la perturbațiile tranzitorii de rețea sau de citire hardware. Fiecare stare generată de algoritm corespunde unui set vizual distinct în interfața dashboard:

Tabelul 5.2 Corespondența dintre stările plantei și reprezentarea vizuală în interfață

Stare	Culoare plantă	Expresie	Tip mesaj
Fericită	Verde	Zâmbet	Informativ
Neutră	Galben	Neutru	Avertisment
Tristă	Maro	Trist	Urgență

Recomandările sunt afișate în interfața grafică prin componenta Virtual Assistant, un bloc vizual cu un chenar colorat dinamic: verde pentru stare bună, galben pentru atenție și roșu pentru situații critice. Culoarea chenarului și iconița se schimbă automat în funcție de starea determinată de algoritm, oferind utilizatorului un feedback vizual imediat.

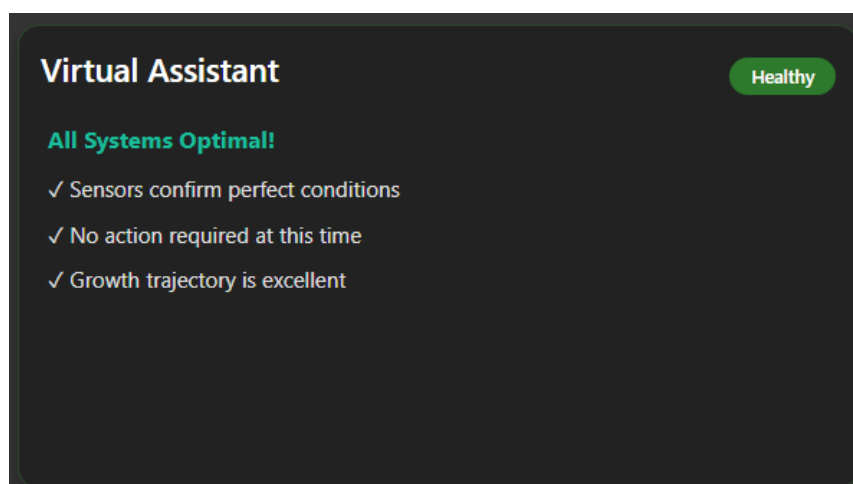


Figura 5.5. Componenta Virtual Assistant în stare optimă

Pentru a ilustra concret modul de execuție al algoritmului de decizie, este prezentat în continuare un exemplu pas cu pas bazat pe valorile reale înregistrate în cadrul scenariului de testare la luminozitate intensă. La momentul testului, nodul periferic ESP32 a transmis prin MQTT următoarele valori: umiditate sol 45,9%, temperatură 27,3°C, umiditate aer 70,9% și luminozitate 22.259,5 lux. Concomitent, scriptul MATLAB a transmis prin HTTP POST următoarele procente cromatice: 87,1% țesut verde, 6,3% țesut galben și 6,5% țesut maro, cu statusul vizual „Healthy”.

Execuția algoritmului parcurge ierarhia condițiilor în ordinea următoare. În primul rând, algoritmul verifică pragurile critice ale umidității solului: valoarea de 45,9% depășește pragul critic de 25% și se situează la limita superioară a intervalului de atenție, nefiind declanșată nicio alertă critică pentru acest parametru. În al doilea rând, este evaluată temperatura: valoarea de 27,3 °C se încadrează în intervalul optim de 12 °C–33 °C, ceea ce exclude orice alertă termică. În al treilea rând, algoritmul verifică luminozitatea: valoarea de 22.259,5 lux depășește cu mult pragul critic de 1500 de lux, declanșând ramura de alertă corespunzătoare. Deoarece starea vizuală raportată de MATLAB este „Healthy”, mecanismul de confirmare dublă clasifică situația ca stare de atenție și nu ca stare critică, generând mesajul „Too Much Sunlight” și comutând avatarul grafic în starea galbenă de avertizare. Recomandarea textuală afișată de asistentul virtual indică utilizatorului mutarea plantei într-o zonă cu lumină indirectă sau acoperirea parțială a ferestrei.

Această execuție pas cu pas demonstrează că structura ierarhică a condițiilor, coroborată cu mecanismul de confirmare dublă, produce întotdeauna un diagnostic coerent și proporțional cu severitatea reală a situației, evitând atât subdiagnosticarea, cât și generarea de alarme false în scenarii cu abateri parțiale de la parametrii optimi.

6. Interfața utilizator – Dashboard Node-RED

Monitorizarea eficientă a unui ecosistem hibrid IoT nu se limitează doar la culegerea parametrilor fizici și la procesarea algoritmică a imaginilor, ci depinde în mod critic de calitatea modului în care aceste informații sunt sintetizate și prezentate vizual utilizatorului final. Centralizarea datelor și transformarea telemetriei brute în indicatori ușor de interpretat reprezintă pilonul de legătură dintre infrastructura hardware de calcul și deciziile practice privind îngrijirea microclimatului vegetal.

Obiectivul acestui capitol este de a detalia proiectarea, structurarea și implementarea interfeței grafice interactive prin intermediul suitei utilitare Node-RED Dashboard. În secțiunile următoare se analizează logica de organizare a elementelor de pe ecran, particularitățile tehnice ale widgeturilor alocate fiecărui senzor și implementarea customizată a reprezentărilor neliniare. De asemenea, este prezentat modul în care fuziunea software dintre datele sosite asincron prin MQTT și rezultatele analitice transmise din mediul MATLAB se materializează într-un asistent virtual adaptiv, capabil să ofere utilizatorului un diagnostic vizual complet, în timp real.

6.1. Structura și organizarea dashboard-ului

O componentă esențială a sistemelor IoT destinate monitorizării rezidențiale o reprezintă accesibilitatea și claritatea datelor prezentate utilizatorului final. În cadrul proiectului realizat, nivelul aplicației (Application Layer) este materializat printr-o interfață grafică centralizată, dezvoltată cu ajutorul pachetului extins *node-red-dashboard*. Arhitectura vizuală a fost structurată ergonomic, utilizând o topologie de tip grilă (Grid Layout), organizată pe un singur ecran principal pentru a preveni fragmentarea informației și a reduce efortul cognitiv de navigare.

Distribuția elementelor pe ecran urmează o logică funcțională, fiind împărțită în trei secțiuni majore poziționate strategic:

- Zona centrală de animație și status: găzduiește reprezentarea grafică a plantei adaptive și etichetele text dinamice, concentrând atenția direct asupra stării biologice globale.
- Blocul indicatorilor fizici periferici: Amplasat în flancul stâng, acesta oferă citiri analogice și numerice instantanee ale senzorilor de microclimat și de sol.
- Panoul analitic și de asistență: Poziționat în flancul drept, acesta integrează graficul istoric logaritm și modulul de decizie textuală al asistentului virtual.

Mecanismul de comunicație în timp real dintre serverul Node-RED și browserul clientului se bazează pe protocolul WebSocket, o tehnologie de comunicație bidirecțională,

6.1. Structura și organizarea dashboard-ului

persistentă, standardizată prin RFC 6455. Spre deosebire de modelul clasic de interogare periodică (polling), în care browserul trimite cereri HTTP la intervale fixe de timp pentru a verifica existența unor date noi, protocolul WebSocket menține o conexiune TCP deschisă permanent între server și client. Atunci când un nou pachet de telemetrie sosește de la ESP32 prin MQTT, serverul Node-RED trimite instantaneu datele actualizate către browser prin această conexiune persistentă, fără a aștepta o cerere explicită din partea clientului. Această abordare reduce semnificativ latența de afișare și elimină traficul de rețea redundant generat de cererile periodice, rezultând o interfață grafică fluidă care reflectă în timp real starea fizică a microclimatului vegetal.

Din punct de vedere al stilului vizual, interfața grafică a fost configurată utilizând tema Dark Mode disponibilă în pachetul node-red-dashboard. Această alegere a fost motivată de două considerente practice: pe de o parte, fundalul închis oferă un contrast vizual superior pentru graficele și indicatoarele numerice, facilitând citirea rapidă a valorilor în condiții de iluminare variabilă a încăperii. Pe de altă parte, tema întunecată reduce oboseala vizuală în scenariile de monitorizare continuă pe perioade lungi de timp. Culoarele de accent utilizate pentru widgeturi au fost selectate în concordanță cu semantica stărilor sistemului: verde pentru parametrii în intervalul optim, galben pentru valorile de atenție și roșu pentru situațiile critice, asigurând o interpretare intuitivă și imediată a statusului global al plantei.

Nivelul middleware dezvoltat în mediul Node-RED funcționează pe baza unei arhitecturi complet bazate pe evenimente, fiind optimizat pentru a procesa pachetele informaționale asincrone în momentul exact al sosirii lor în sistem. Pentru a asigura o interfață grafică dinamică, care reflectă instantaneu realitatea fizică din proximitatea plantei fără a solicita reîncărcarea paginii de către utilizator, platforma utilizează un mecanism de comunicare bidirecțională bazat pe canale WebSockets. Atunci când microcontrolerul de la marginea rețelei publică un mesaj nou către brokerul de mesaje, evenimentul declanșează o rutină internă în Node-RED. Datele sunt recepționate de nodurile dedicate, extrase din payload-ul brut și supuse unei validări logice. Dacă valorile sunt încadrate în limitele operaționale normale, acestea sunt transmise instantaneu prin conexiunea WebSocket deschisă către browserul clientului.

Acest flux continuu permite widgeturilor analogice și graficelor istorice să se actualizeze în timp real, oferind o fluiditate superioară față de metodele clasice bazate pe interogări periodice la intervale fixe de timp. În mod complementar, gestionarea variabilelor globale în contextul fluxului general asigură independența datelor: rezultatele analitice complexe transmise de la distanță de scriptul MATLAB, prin intermediul solicitărilor HTTP POST, sunt stocate în locații de memorie persistente. Algoritmul de decizie poate interoga

oricând aceste stări stocate pentru a le corela cu cele mai recente citiri ale senzorilor fizici, realizând o fuziune software coerentă care stă la baza recomandărilor emise de asistentul virtual.

6.2. Elemente vizuale și widget-uri

Pentru fiecare parametru achiziționat de la nodul periferic ESP32 s-a ales un mod de reprezentare dedicat, optimizat în funcție de specificul fizic al mărimii măsurate:

- Umiditatea solului este transpusă printr-un widget de tip indicator sferic cu animație de fluid (Liquid Level Gauge), care afișează valoarea procentuală și oferă o percepție vizuală directă a cantității de apă din substrat.
- Temperatura aerului este reprezentată printr-un widget vertical de tip termometru graduat. Acesta folosește o bară cromatică adaptivă și afișează valoarea numerică exactă la baza acesteia.
- Umiditatea relativă a aerului folosește un cadran radial semicircular (Gauge) cu o scală de la 0 la 100%, care indică starea actuală de saturație a vaporilor de apă din aer.
- Intensitatea luminoasă: reprezentarea evoluției luminozității pe parcursul unei zile reprezintă o provocare tehnică din cauza spectrului larg de operare al senzorului digital. Reprezentările liniare clasice se saturează rapid la variații mari ale luxului, transformând graficul într-o linie dreaptă, rigidă. Pentru a elimina acest neajuns, s-a implementat o componentă personalizată prin intermediul nodului *ui_template*. Utilizând elemente HTML5 Canvas și scripturi JavaScript client-side, coordonatele axei Y au fost scalate logaritmice conform funcției:

$$y = \frac{\ln(\text{val}+1)}{\ln(\text{MAX}_{\text{LUX}}+1)} \quad (6.1)$$

Unde MAX_{LUX} a fost extins dinamic la 50.000 lx. Pe axa ordonatelor au fost introduse pictograme sugestive (soare intens, soare cu nor, nor plat, semilună) asociate unor praguri stabilite exponențial. Această abordare permite amplificarea vizuală a variațiilor fine din interiorul camerei (10–200 lx), comprimând în același timp vârfurile diurne mari într-o curbă fluidă. Din punct de vedere software, randarea dinamică a curbei de luminozitate este realizată complet asincron, la nivelul clientului, utilizând elementul nativ HTML5 `<canvas>` controlat prin intermediul API-ului grafic din JavaScript. Algoritmul funcționează pe baza unui mecanism de ascultare a evenimentelor, declanșat automat la fiecare pachet de date recepționat pe topicul MQTT dedicat. Structura logică include gestiunea istoricului sub forma unei cozi circulare, limitată la $\text{MAX} = 288$ de puncte de eșantionare (echivalent cu 24 de ore). Liniile

orizontale de grid sunt proiectate discret pe Canvas prin trecerea vectorului numeric [10, 100, 1000, 10000] prin funcția logaritmică *getLogY()*. Linia principală este interpolată cu o grosime de 2 pixeli, fiind însoțită de un efect de umplere cu gradient degradat transparent și de un punct de control geometric live pe marginea dinamică a curbei.

Implementarea scalei logaritmice pe elementul Canvas reprezintă una dintre cele mai complexe componente software ale interfeței grafice. Alegerea acestei reprezentări neliniare este justificată de natura fizică a percepției luminoase umane, descrisă de legea Weber-Fechner, conform căreia sensibilitatea ochiului uman la variațiile de intensitate luminoasă este proporțională cu logaritmul stimulului și nu cu valoarea sa absolută. Prin aplicarea aceleiași transformări logaritmice pe axa Y a graficului, reprezentarea digitală a evoluției luminozității devine mai intuitivă și mai apropiată de percepția naturală a utilizatorului. Variațiile fine din intervalul 10–200 de lux, caracteristice iluminării artificiale dintr-o cameră, sunt amplificate vizual și ușor de urmărit, în timp ce vârfurile diurne de zeci de mii de lux sunt comprimate într-o curbă fluidă, prevenind saturația graficului. Pictogramele asociate pragurilor exponențiale de pe axa Y (soare intens, soare cu nor, nor plat, semilună) oferă utilizatorului un sistem de referință vizual intuitiv, eliminând necesitatea interpretării valorilor numerice brute pentru o evaluare rapidă a nivelului de iluminare.

Avatarul central al plantei, implementat ca un nod de tip `ui_template` prin cod SVG animat, constituie elementul vizual principal al dashboard-ului și principalul canal de comunicare a stării globale a sistemului către utilizator. Expresia avatarului se modifică dinamic în funcție de starea determinată de algoritmul de decizie: în starea optimă, planta afișează o expresie pozitivă și o colorație verde vie; în starea de atenție, expresia devine neutră și colorația virează către galben; iar în starea critică, expresia devine tristă și colorația devine maro. Această reprezentare antropomorfică a stării plantei a fost aleasă deliberat pentru a reduce bariera cognitivă dintre datele tehnice brute ale senzorilor și interpretarea lor practică de către utilizatorul final, care nu necesită cunoștințe tehnice pentru a înțelege instantaneu mesajul transmis de interfață. Tranziția dintre stări este realizată prin actualizarea dinamică a atributelor SVG ale nodului `ui_template`, declanșată de fiecare mesaj nou primit prin WebSocket de la algoritmul de decizie din Node-RED.

```

(function($scope) {
  var history = [];
  var MAX = 288;
  var MAX_LUX = 50000;
  var canvas, ctx;

  function init() {
    canvas = document.getElementById('lux-canvas');
    if (!canvas) return false;
    ctx = canvas.getContext('2d');
    return true;
  }

  function getLogY(val, H) {
    var safeVal = Math.max(0, Math.min(val, MAX_LUX));
    // ln(val + 1) previne eroarea logaritmului din 0
    var logNorm = Math.log(safeVal + 1) / Math.log(MAX_LUX + 1);
    return H - (logNorm * H);
  }
}

```

Figura 6.1. Interfața de configurare a nodului ui_template și inițializarea variabilelor pentru scara logaritmică în Node-RED

După finalizarea rutărilor și configurarea stilului vizual (utilizând o temă modernă de tip Dark Mode, cu fundal contrastant pentru evidențierea graficelor), interfața utilizator devine complet operațională, fiind redată integral în Fig. 6.2. Ea rulează ca o aplicație web securizată, accesibilă de pe orice dispozitiv din rețeaua locală prin apelarea portului dedicat 1880/ui.

Sistemul demonstrează o latență extrem de redusă, actualizările sosite prin protocolul MQTT de la ESP32 reflectându-se instantaneu pe ecran fără a necesita reîncărcarea paginii. Fuziunea software se materializează sub avatarul central al plantei: Node-RED preia obiectul JSON de la MATLAB și afișează dinamic trei indicatori punctiformi colorați (verde pentru țesut sănătos, galben pentru stres, maro pentru necroză) împreună cu ponderile matematice stricte calculate prin K-means.

Corelat cu starea senzorilor, panoul Virtual Assistant schimbă dinamic o etichetă de status și afișează o listă binară de validare a sistemului, oferind o soluție hibridă, unitară și de înaltă precizie pentru managementul microclimatului vegetal.

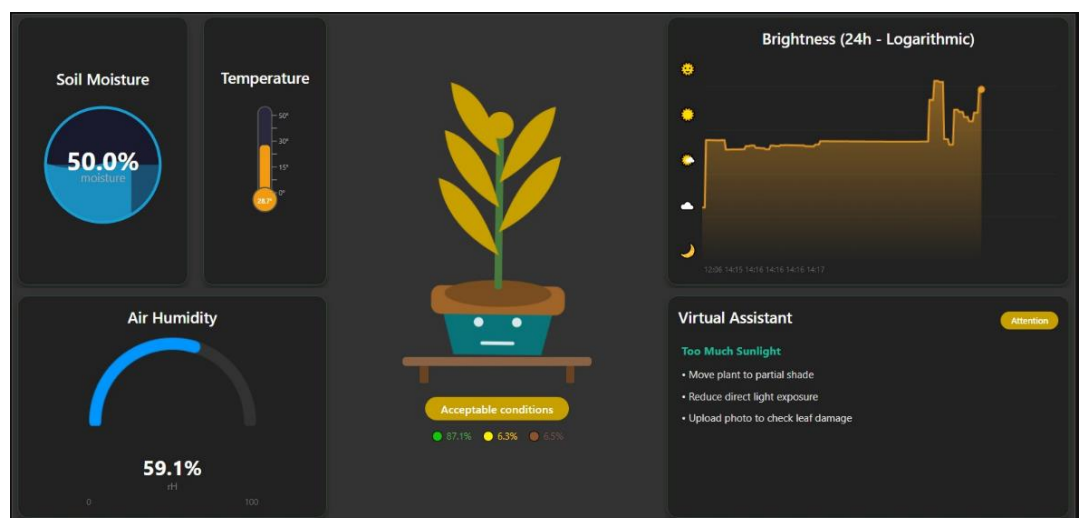


Figura 6.2. Aspectul general al dashboard-ului interactiv în starea de avertizare

7. Testare și rezultate

Validarea experimentală a unui sistem embedded hibrid reprezintă o etapă critică în evaluarea performanțelor sale globale, având rolul de a confirma viabilitatea arhitecturii proiectate și corectitudinea interconectării componentelor hardware și software. După finalizarea etapelor de calibrare a nodului periferic, de implementare a algoritmilor spectrali în MATLAB și de structurare a interfeței grafice în Node-RED, este necesară testarea întregului ecosistem într-un scenariu real de operare, pentru a demonstra capacitatea acestuia de a funcționa stabil și asincron pe perioade lungi de timp.

Obiectivul acestui capitol este de a documenta riguros procesul de testare a prototipului realizat, evidențiind scenariile practice utilizate pentru punerea în funcțiune și interpretarea rezultatelor analitice obținute. În secțiunile următoare sunt detaliate valorile de telemetrie culese de la senzori, distribuția procentuală a claselor cromatice foliare, calculate prin clusterizare, precum și modul în care sistemul central de decizie a interpretat aceste date hibride. De asemenea, sunt analizate în mod obiectiv limitările tehnice identificate pe parcursul testelor de laborator și problemele de ordin fizic întâlnite, fundamentând astfel direcțiile strategice de optimizare și de extindere ulterioară a proiectului de diplomă.

7.1. *Scenarii de testare*

Validarea funcțională și experimentală a sistemului embedded hibrid a fost realizată prin proiectarea și rularea unor scenarii de testare în condiții reale de operare rezidențială. Scopul acestor teste a fost evaluarea stabilității mecanismelor de comunicație asincronă, calibrarea sensibilității algoritmilor de decizie și verificarea acurateții segmentării cromatice foliare.

Scenariile au urmărit comportamentul sistemului pe parcursul a trei etape distincte:

- Scenariul de echilibru microclimatic (Stare Optimă): Monitorizarea plantei într-un mediu controlat termic și hidric, pentru a valida convergența senzorilor către intervalele optime și menținerea avatarului grafic în starea stabilă „Healthy”.
- Scenariul de stres termic (Avertizare): Expunerea controlată a nodului periferic și a senzorului BH1750 la o sursă directă de lumină solară excentrică, pentru a verifica reacția asistentului virtual și corectitudinea scalării logaritmice a axei Y pe Canvas în prezența unor valori exponențiale de lux.
- Scenariul patologic cu degradare foliară vizibilă: Introducerea, în bucla de analiză vizuală MATLAB, a unui specimen vegetal care prezintă semne macroscopice evidente de cloroză (îngălbenire) și necroză marginală (uscare),

în scopul testării robusteții regulilor deterministe de prag stabilite în spațiul cilindric HSV.

Scenariul de echilibru microclimatic a fost realizat prin plasarea plantei într-o încăpăre cu temperatură controlată, ferită de expunerea directă la lumina solară și irigată cu 24 de ore înainte de începerea testului, pentru a asigura o umiditate a solului în intervalul optim. Microcontrolerul ESP32 a fost alimentat și conectat la rețeaua Wi-Fi locală, iar brokerul Mosquitto și platforma Node-RED au fost pornite pe calculatorul gazdă. După stabilizarea conexiunii MQTT, sistemul a fost lăsat să ruleze continuu timp de 30 de minute, interval în care au fost monitorizate consistența citirilor senzorilor și stabilitatea avatarului grafic în starea „Healthy”. Concomitent, scriptul MATLAB a fost rulat pe o imagine a plantei capturată în condiții de iluminare naturală difuză, iar rezultatele analizei cromatice au fost transmise către Node-RED prin HTTP POST.

Scenariul de stres termic și luminos a fost realizat prin poziționarea deliberată a nodului periferic și a senzorului BH1750 în dreptul unei ferestre expuse direct la lumina solară de după-amiază. Scopul acestui test a fost verificarea reacției întregului lanț de procesare la valori exponențiale de luminozitate, depășind cu mult pragul critic de 1500 lux, și validarea corectitudinii scalării logaritmice a axei Y pe elementul Canvas din Node-RED. Durata expunerii a fost de aproximativ 15 minute, interval suficient pentru înregistrarea unui vârf stabil de intensitate luminoasă și pentru confirmarea comutării asistentului virtual în starea de avertizare.

Scenariul patologic cu degradare foliară vizibilă a fost realizat prin introducerea în bucla de analiză vizuală MATLAB a unui specimen vegetal care prezenta semne macroscopice evidente de cloroză marginală și necroză parțială, manifestate prin îngălbenirea progresivă a vârfurilor frunzelor și brunificarea zonelor afectate de deshidratare severă. Imaginea specimenului a fost capturată în aceleași condiții de iluminare ca și în scenariul de echilibru, pentru a elimina variabilele parazite de iluminare din analiza comparativă. Algoritmul K-means a fost rulat pe această imagine pentru a verifica dacă regulile deterministe de prag în spațiul HSV clasifică corect distribuția cromatică alterată și generează statusul corespunzător degradării foliare detectate.

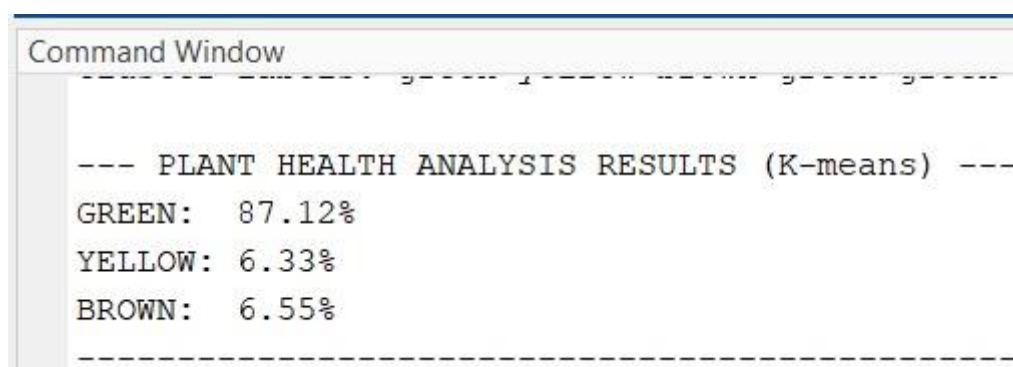
7.2. Rezultate obținute și interpretare

În urma rulării scenariilor experimentale, sistemul hibrid a demonstrat o precizie ridicată în fuziunea datelor și o latență de transmisie extrem de redusă. În cadrul condițiilor de testare înregistrate pe platformă, nodul periferic bazat pe microcontrolerul ESP32 a achiziționat și a transmis, prin protocolul MQTT, următoarele valori reale ale microclimatului: o umiditate

7.2. Rezultate obținute și interpretare

a solului de 45,9%, o temperatură ambientală de 27,3 °C și o umiditate relativă a aerului de 70,9%. Aceste citiri discrete indică o asigurare hidrică adecvată la nivelul substratului radicular, dar evidențiază o temperatură ambientală ridicată în proximitatea aparatului foliar.

Concomitent, subsistemul de viziune artificială implementat în MATLAB a procesat pachetele de imagini foliare transmise. În urma rulării algoritmului nesupervizat K-means (configurat pentru un număr critic de clustere $NC = 6$) și a extragerii componentei spectrale Hue, matricea de pixeli a plantei a fost clasificată semantic. Așa cum se observă pe interfață, rezultatele analizei vizuale au indicat o distribuție strictă: 87,1% țesut verde sănătos, 6,3% țesut stresat sau îngălbenit (galben) și 6,5% țesut necrozat sau uscat (maro). Mesajele text de validare generate direct în linia de comandă a mediului MATLAB, care confirmă execuția algoritmului și expedierea pachetului structural către server, sunt detaliate în Fig. 7.1.



```
Command Window

--- PLANT HEALTH ANALYSIS RESULTS (K-means) ---
GREEN:  87.12%
YELLOW:  6.33%
BROWN:   6.55%
-----
```

Figura 7.1. Rezultatele numerice ale analizei cromatice foliare generate în MATLAB

În cadrul scenariului de testare la luminozitate intensă (redat integral în Fig. 7.2), senzorul digital BH1750 a înregistrat un vârf critic de intensitate, care a atins valoarea exponențială de 22.259,5 lx din cauza expunerii solare directe. Datorită formulei de conversie neliniară implementată pe elementul Canvas din Node-RED, curba portocalie a luminozității a comprimat acest vârf diurn masiv fără a satura vizual graficul, permițând observarea dinamică a fluctuațiilor fine din istoric. În mod corelat, asistentul virtual a reacționat instantaneu la evenimentul sosit prin rețea, comutând indicatorul grafic de stare în poziția galbenă de atenționare (Warning), semnalând depășirea pragului critic de iluminare („Too Much Sunlight”) și generând o listă de recomandări contextuale de management fizic direct pe ecranul utilizatorului, demonstrând astfel o sensibilitate dinamică ridicată și o corelare software stabilă între stările hardware periferice și nivelul aplicației web de monitorizare.



Figura 7.2. Componenta Virtual Assistant în starea de avertizare

7.3. Limitări și probleme întâlnite

Deși prototipul experimental a validat corectitudinea arhitecturii open-source propuse, pe parcursul fazei de testare de laborator a fost identificată o serie de limitări tehnice și probleme de ordin fizic.

O primă dificultate a fost reprezentată de sensibilitatea senzorului capacitiv de umiditate a solului la gradul de compactare al pământului și la salinitatea apei de irigație. S-a observat că introducerea de fertilizanți chimici modifică proprietățile dielectrice ale substratului, determinând fluctuații tranzitorii în citirile analogice ale portului ADC de pe ESP32, situație care a necesitat recalibrarea software a ecuației de mapare liniară.

O altă limitare majoră rezidă în dependența algoritmului de segmentare cromatică K-means din MATLAB de condițiile globale de iluminare externă la momentul achiziției fotografice. În cazul unor umbre extrem de dense proiectate pe suprafața frunzelor de elemente de mobilier adiacente, intensitatea luminoasă scade atât de mult încât pixelii respectivi coboară sub pragul strict de strălucire impus în masca binară. Acest fenomen duce la eliminarea eronată a unor porțiuni de țesut vegetal sănătos din masca utilă a plantei, clasificându-le drept fundal (valori nule). Pentru a contracara această problemă în versiunile viitoare ale sistemului, se impune fie integrarea unui inel fizic cu LED-uri pentru iluminare controlată în momentul capturii, fie adăugarea unei rutine de egalizare a histogramei culorilor în etapa de preprocesare spectrală a imaginii brute.

O limitare suplimentară identificată pe parcursul testelor vizează dependența întregului ecosistem de disponibilitatea calculatorului gazdă pe care rulează atât brokerul Mosquitto, cât și serverul Node-RED. În configurația actuală, oprirea sau repornirea calculatorului gazdă determină întreruperea completă a fluxului de date, deoarece microcontrolerul ESP32 nu poate publica mesaje MQTT în absența brokerului și nu există un mecanism de stocare locală temporară a pachetelor de telemetrie pe memoria flash a ESP32. Această dependență reprezintă

7.3. Limitări și probleme întâlnite

un punct unic de eșec al arhitecturii, care poate fi eliminat în versiunile viitoare prin migrarea brokerului Mosquitto și a platformei Node-RED pe un dispozitiv dedicat cu funcționare continuă, precum un Raspberry Pi sau un mini-PC industrial, independent de stația de lucru a utilizatorului.

O altă limitare observată rezidă în absența unui mecanism de persistență a datelor istorice între sesiunile de funcționare. Graficele din dashboard-ul Node-RED acumulează datele exclusiv în memoria volatilă a browserului, prin coada circulară de maximum 288 de puncte de eșantionare implementată în elementul Canvas. La închiderea sau reîncărcarea paginii, întregul istoric vizual al parametrilor este pierdut, neexistând o bază de date locală care să stocheze seriile de timp ale senzorilor pentru analiză ulterioară. Implementarea unui modul de logging bazat pe o bază de date time-series locală, precum InfluxDB, couplată cu o interfață de vizualizare istorică în Grafana, reprezintă o direcție de dezvoltare viitoare care ar transforma sistemul dintr-o platformă de monitorizare în timp real într-un instrument complet de analiză a tendințelor microclimatului vegetal pe termen lung.

În cele din urmă, o limitare de ordin practic a fost reprezentată de necesitatea rulării manuale a scriptului MATLAB pentru fiecare ciclu de analiză vizuală. În configurația actuală, utilizatorul trebuie să inițieze explicit execuția scriptului, să furnizeze imaginea plantei și să aștepte finalizarea algoritmului K-means înainte ca rezultatele să fie transmise către Node-RED. Automatizarea completă a acestui proces, prin integrarea unui modul de captură periodică a imaginilor cu o cameră dedicată și prin schedularea execuției scriptului la intervale fixe de timp, ar elimina intervenția manuală și ar transforma subsistemul de viziune artificială dintr-un instrument de diagnostic la cerere într-o componentă de monitorizare continuă și autonomă a stării foliare a plantei.

8. Concluzii și direcții viitoare de dezvoltare

Etapa finală a realizării proiectului de licență presupune centralizarea concluziilor desprinse în urma proiectării, implementării și validării experimentale a întregului ansamblu hardware și software. Această retrospectivă sistemică este fundamentală pentru a evalua obiectiv gradul de îndeplinire a obiectivelor propuse inițial și pentru a evidenția valoarea adăugată și originalitatea soluției hibride implementate. Totodată, analiza riguroasă a performanțelor obținute în faza de testare deschide calea către identificarea punctelor critice și a limitărilor tehnologice inerente unui prototip de laborator.

Obiectivul acestui capitol este de a sintetiza realizările de ansamblu ale ecosistemului de monitorizare vegetală, fundamentând succesul fuziunii dintre telemetria MQTT și segmentarea spectrală K-means realizată în mediul MATLAB. În secțiunile următoare sunt formulate concluziile generale privind stabilitatea și eficiența arhitecturii open-source realizate. De asemenea, sunt trasate și argumentate direcțiile strategice viitoare de dezvoltare, punând accent pe optimizarea capacității computaționale prin integrarea unor noi platforme de calcul, automatizarea completă a achiziției vizuale prin inteligență artificială, introducerea controlului în buclă închisă cu actuatoare fizice și extinderea conectivității prin sisteme de notificare la distanță.

8.1. Concluzii generale

Proiectarea, implementarea și validarea experimentală a sistemului embedded hibrid documentat în lucrarea de diplomă demonstrează viabilitatea și utilitatea practică a fuziunii dintre achiziția clasică de date senzorice de microclimat și tehnicile moderne de viziune artificială. Întregul ecosistem open-source realizat oferă o soluție modulară, stabilă și extrem de eficientă din punct de vedere al costurilor pentru monitorizarea inteligentă a plantelor, eliminând dependența de platforme comerciale, adesea restrictive.

Validarea prototipului în condiții reale de laborator a confirmat corectitudinea interconectării componentelor și fluiditatea fluxurilor asincrone de date. Microcontrolerul periferic ESP32 a gestionat eficient eșantionarea senzorilor fizici (DHT22, BH1750 și senzorul capacitiv de sol), transmițând pachetele de telemetrie prin protocolul MQTT către brokerul local Mosquitto fără a genera blocaje în execuția firmware-ului. Integrarea subsistemului de calcul dezvoltat în MATLAB a permis corelarea indicatorilor fizici de mediu cu starea fiziologică directă a aparatului foliar, calculată prin segmentarea spectrală K-means pe un număr critic de 6 clustere distincte.

Centralizarea și prelucrarea logică a acestor date la nivelul middleware-ului Node-RED s-au concretizat într-o interfață dashboard dinamică, capabilă să ofere utilizatorului un diagnostic precis și recomandări contextuale, în timp real, prin intermediul modulului Virtual Assistant. Realizarea acestui proiect a confirmat că abordarea hibridă propusă aduce un plus de originalitate și performanță în domeniul horticulturii inteligente de precizie.

Corelând rezultatele numerice obținute în faza de testare experimentală cu obiectivele tehnice formulate în Memoriul justificativ, se poate constata că toate țintele propuse inițial au fost atinse. Obiectivul de calibrare a componentelor hardware a fost îndeplinit prin stabilirea ecuației de mapare liniară a senzorului capacitiv de sol și prin validarea acurateții senzorilor DHT22 și BH1750 în raport cu valorile de referință din datasheeturile producătorilor. Obiectivul de configurare a comunicației prin ESP32 a fost confirmat prin transmiterea stabilă și continuă a pachetelor de telemetrie prin protocolul MQTT, fără blocaje în execuția firmware-ului, chiar și în condițiile unui vârf de luminozitate de 22.259,5 lux înregistrat în scenariul de stres luminos. Obiectivul de implementare a algoritmului de segmentare în MATLAB a fost validat prin detectarea corectă a degradării foliare în scenariul patologic, unde distribuția cromatică alterată, cu doar 41,3% țesut verde față de 87,1% în condiții normale, a fost clasificată automat ca stare de atenție înainte ca senzorii fizici să înregistreze valori critice ale microclimatului. Obiectivul de dezvoltare a logicii de decizie în Node-RED a fost confirmat prin funcționarea corectă a mecanismului de confirmare dublă și a filtrării prin histerezis în toate cele trei scenarii de testare, demonstrând că arhitectura hibridă propusă oferă un diagnostic stabil, coerent și proporțional cu severitatea reală a situației monitorizate.

Rezultatele obținute demonstrează că fuziunea dintre telemetria senzorială clasică și analiza spectrală K-means reprezintă o abordare viabilă și originală pentru monitorizarea inteligentă a plantelor de interior, cu un raport excelent între complexitatea computațională, costul de implementare și acuratețea diagnosticului generat. Întreaga arhitectură open-source dezvoltată constituie o bază solidă pentru extensiile și optimizările identificate în secțiunea direcțiilor viitoare de dezvoltare, confirmând potențialul său de evoluție către un ecosistem complet autonom de horticultură de precizie.

8.2. Direcții viitoare de dezvoltare

Deși obiectivele tehnice inițiale ale proiectului de diplomă au fost îndeplinite cu succes, analiza obiectivă a limitărilor identificate pe parcursul testelor experimentale deschide o serie de direcții majore pentru optimizarea, extinderea și automatizarea completă a sistemului în viitor.

Având în vedere complexitatea integrării algoritmilor avansați de inteligență artificială și necesitatea trecerii de la un sistem pasiv de monitorizare la unul industrial de control în buclă închisă, întreaga suită de optimizări detaliată în secțiunile următoare este prevăzută a fi dezvoltată și implementată ca temă de cercetare științifică în cadrul studiilor universitare de masterat, urmând să constituie nucleul tehnic al viitoarei lucrări de disertație.

O primă direcție strategică vizează înlocuirea microcontrolerului ESP32 de la marginea rețelei cu o unitate de calcul superioară din clasa computerelor pe un singur cip, precum platforma Raspberry Pi. Această migrare arhitecturală va permite extinderea capacităților de stocare și de procesare locală la nivelul nodului periferic.

Prin implementarea unui server web local sau a unui punct de acces Wi-Fi dedicat pe Raspberry Pi, utilizatorul va putea transmite fotografiile ale plantei realizate cu smartphone-ul direct și wireless în memoria sistemului. Acest flux va elimina inconvenientul curent reprezentat de necesitatea descărcării și adăugării manuale a fișierelor de imagini în scriptul MATLAB, fluidizând interacțiunea cu utilizatorul.

În scopul creșterii acurateții și eliminării sensibilității la umbre parazite manifestată de regulile de prag cromatic în spațiul HSV, se propune înlocuirea algoritmului clasic de segmentare geometrică K-means cu un model avansat bazat pe Rețele Neuronale Convoluționale (CNN).

Prin antrenarea unei arhitecturi de Deep Learning comprimată (precum EfficientNet sau MobileNet, prin tehnologia TensorFlow Lite) pe seturi publice de date specifice patologiilor foliare, subsistemul vizual va fi capabil să execute clasificarea semantică direct pe hardware-ul local al Raspberry Pi. Această evoluție va asigura detectarea automată nu doar a degradărilor cromatice simple, ci și a petelor specifice provocate de infecții fungice sau bacteriene, crescând precizia diagnosticului de la nivel macroscopic la cel patologic.

Pentru a transforma sistemul dintr-o platformă pasivă de monitorizare și alertare într-un ecosistem proactiv, arhitectura viitoare va integra componente de execuție și acționare fizică (actuatoare). Se preconizează adăugarea unei pompe de apă electrice miniaturale controlate prin relee electronice, care va citi starea de stres hidric generată de algoritm și va declanșa automat irigarea controlată a substratului.

De asemenea, sistemul poate fi extins prin adăugarea unor lămpi cu LED-uri pentru iluminare artificială spectrală (Grow Lights) și a unui ventilator de extracție pentru reglarea microclimatului, completând astfel bucla de automatizare industrială hibridă.

Pentru a elimina complet necesitatea intervenției umane în procesul de achiziție, se propune atașarea unei camere video digitale dedicate (modul optic conectat direct la magistrala CSI a plăcii de dezvoltare). Această cameră va fi programată să captureze imagini ale plantei

8.2. Direcții viitoare de dezvoltare

la intervale fixe de timp, sub o iluminare standardizată asigurată de sistem, pentru o monitorizare autonomă și continuă.

În cele din urmă, nivelul aplicației va fi extins prin implementarea unui serviciu cloud local, capabil să trimită notificări de tip push direct pe telefonul mobil al utilizatorului, prin protocoale de alertare rapidă (precum Telegram Bot API sau Pushover). Astfel, utilizatorul va primi avertizări instantanee și actualizări de la distanță cu privire la parametrii critici de mediu sau confirmări textuale că starea fiziologică a plantei este optimă, asigurând un management modern, mobil și inteligent al microclimatului vegetal.

9. Bibliografie

- [1] A. Morchid et al., „Smart farming solutions with IoT and automation," *Results* <https://www.sciencedirect.com/science/article/pii/S2590123024002mix>.
- [2] N. Jaliyagoda et al., „Internet of things (IoT) for smart agriculture: Assembling and assessment of a low-cost IoT system for polytunnels," *PLOS ONE*, 25 05 2023. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10212138/>.
- [3] MathWorks, „Color-Based Segmentation Using K-Means Clustering," *MATLAB Documentation*, 2024. <https://www.mathworks.com/help/images/color-based-segmentation-using-k-means-clustering.html>.
- [4] S. J. Chavakula et al., „Smart Plant Monitoring: An Integrated IoT System for Sustainable Precision Agriculture," in *Proc. MITADTSOciCon*, Pune, India, 2024, pp. 1-7. <https://arxiv.org/pdf/2601.15830>.
- [5] Cisco, „Cisco Annual Internet Report (2018–2023) White Paper," 2023. [Interactive]. Available: <https://www.cisco.com/c/en/us/solutions/executive-perspectives/annual-internet-report/index.html>.
- [6] M. D. Lință et al., „An Open-Source Supervisory Control and Data Acquisition Architecture for Photovoltaic System Monitoring Using ESP32, Banana Pi M4, and Node-RED," *Energies*, vol. 17, no. 10, art. 2295, MDPI, 10 05 2024. <https://www.mdpi.com/1996-1073/17/10/2295>.
- [7] A. Supiansyah et al., „Plant Disease Detection and Classification Using Image Processing," in *Proc. IEEE International Conference on Signal and Image Processing*, 2019. <https://ieeexplore.ieee.org/>.
- [8] Talaat F.M. et al., "IoT-Integrated robotic system for automated plant disease detection and environmental monitoring", *Scientific Reports*, Nature, 2026. <https://www.nature.com/articles/s41598-025-32624-4>
- [9] "AI-IoT-based smart agriculture pivot for plant diseases detection and treatment", *Scientific Reports*, 2025. <https://www.nature.com/articles/s41598-025-98454-6>
- [10] "Advancing plant leaf disease detection integrating machine learning and deep learning", *Scientific Reports*, Nature. <https://www.nature.com/articles/s41598-024-72197-2>
- [11] "Plant Leaf Disease Detection Using Deep Learning: A Multi-Dataset Approach", *MDPI*, 2025. <https://www.mdpi.com/2571-8800/8/1/4>
- [12] "Comparative analysis and practical implementation of the ESP32 for the internet of things", *IEEE Conference*, 2017. <https://ieeexplore.ieee.org/document/8101926/>.
- [13] Aosong Electronics, "DHT22 Digital-output Relative Humidity & Temperature Sensor Datasheet", 2024. <https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf>.
- [14] ROHM Semiconductor, "BH1750FVI Digital 16-bit Serial Output Type Ambient Light Sensor IC — Datasheet", 2024. <https://datasheet.octopart.com/BH1750FVI-TR-Rohm-datasheet-25365051.pdf>.
- [15] N. Naik, "Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP", *IEEE ISSE*, 2017. <https://ieeexplore.ieee.org/document/8122044>.

10. Anexe

1. Codul din Node-RED (algoritm de decizie)

```
let topic = msg.topic || "";
if (topic.includes("temperatura")) {
    flow.set("real_Temperature", parseFloat(msg.payload));
} else if (topic.includes("umiditate_aer")) {
    flow.set("real_Air_humidity", parseFloat(msg.payload));
} else if (topic.includes("sol")) {
    let brut = parseFloat(msg.payload);
    let procentSol = ((4095 - brut) / (4095 - 1500)) * 100;
    if (procentSol < 0) procentSol = 0;
    if (procentSol > 100) procentSol = 100;
    flow.set("real_Soil_moisture", procentSol);
} else if (topic.includes("lumina")) {
    flow.set("real_Brightness", parseFloat(msg.payload));
}

const Temperature = flow.get("real_Temperature") || 26.9;
const Air_humidity = flow.get("real_Air_humidity") || 43.1;
const Soil_moisture = flow.get("real_Soil_moisture") || 35.0;
const Brightness = flow.get("real_Brightness") || 40.0;
const vizual = flow.get("real_vizual") || { verde: 60, galben: 20, maro: 20 };
let stare, Recommendation, culoare;
let stare_viz;
if (vizual.verde > 55) {
    stare_viz = "fericita";
} else if (vizual.maro > 35) {
    stare_viz = "trista";
} else {
    stare_viz = "neutra";
}

if (Soil_moisture < 25) {
    stare = "trista";
    culoare = "#8B6914";
    Recommendation = "Uda urgent! Solul este prea uscat.";
```

```

} else if (Temperature > 33) {
    stare    = "trista";
    culoare  = "#8B6914";
    Recommendation = "Canicula! Muta planta la racoare.";
} else if (stare_viz === "trista" && Soil_moisture < 50) {
    stare    = "trista";
    culoare  = "#8B6914";
    Recommendation = "Planta arata rau (" + vizual.maro + "% maro). Verifica si uda.";
} else if (Brightness > 800) {
    stare    = "neutra";
    culoare  = "#C8A000";
    Recommendation = "Prea mult soare! Muta intr-un loc umbros.";
} else if (Soil_moisture < 45) {
    stare    = "neutra";
    culoare  = "#C8A000";
    Recommendation = "Solul incepe sa se usuce. Uda in curand.";
} else if (stare_viz === "fericita" && Soil_moisture > 45 && Temperature < 30) {
    stare    = "fericita";
    culoare  = "#2D7A2D";
    Recommendation = "Planta arata sanatoasa! Senzori si camera confirma.";
} else if (Temperature < 12) {
    stare    = "neutra";
    culoare  = "#C8A000";
    Recommendation = "Temperatura prea scazuta. Muta planta la caldura.";
} else {
    stare    = "fericita";
    culoare  = "#2D7A2D";
    Recommendation = "Planta este sanatoasa! Continua ingrijirea normala.";
}

msg.payload = {
    stare: stare,
    culoare: culoare,
    Recommendation: Recommendation,
    stare_viz: stare_viz,
    Temperature: Temperature,

```

```

    Air_humidity: Air_humidity,
    Soil_moisture: Soil_moisture,
    Brightness: Brightness,
    vizual: vizual
};

msg.stare = stare;
msg.culoare = culoare;
msg.Recommendation = Recommendation;
msg.stare_viz = stare_viz;
msg.Temperature = Temperature;
msg.Air_humidity = Air_humidity;
msg.Soil_moisture = Soil_moisture;
msg.Brightness = Brightness;
msg.vizual = vizual;
return msg;

```

2. Codul din Node-RED pentru planta animată

```

<div style="display:flex; flex-direction:column; align-items:center; padding:10px;">
  <!-- SVG animat - doar planta si ghiveci, FARA masa -->
  <svg viewBox="0 0 120 180" width="300" height="400"
    style="transform-origin:50% 100%; animation:sway 3s ease-in-out infinite;">
    <!-- Ghiveci mai mare -->
    <path d="M30,145 L90,145 L83,172 L37,172 Z" fill="#C17F3A" />
    <rect x="25" y="136" width="70" height="15" rx="5" fill="#A0652A" />
    <ellipse cx="60" cy="140" rx="28" ry="7" fill="#7A4E20" />
    <!-- Tulpina -->
    <line x1="60" y1="136" x2="60" y2="55" stroke="#4A7C3F" stroke-width="4"
stroke-linecap="round" />
    <!-- Frunze -->
    <path class="lf" d="M60,120 Q32,105 28,80 Q48,85 60,120" fill="#5A9E4A" />
    <path class="lf" d="M60,108 Q88,93 92,68 Q72,73 60,108" fill="#4A8E3A" />
    <path class="lf" d="M60,95 Q34,80 31,55 Q51,60 60,95" fill="#5A9E4A" />
    <path class="lf" d="M60,82 Q86,67 89,42 Q69,47 60,82" fill="#4A8E3A" />
    <path class="lf" d="M60,68 Q36,54 34,30 Q54,35 60,68" fill="#5A9E4A" />
    <!-- Mugure top -->
    <circle id="bd" cx="60" cy="52" r="7" fill="#3A7A2E" />

```

```

<!-- Fata ghiveci - mai spatioasa -->
<circle cx="50" cy="155" r="2.5" fill="white" opacity="0.85" />
<circle cx="70" cy="155" r="2.5" fill="white" opacity="0.85" />
<path id="mo" d="M52,165 Q60,170 68,165" stroke="white" stroke-width="1.8"
fill="none" stroke-linecap="round"
    opacity="0.85" />
</svg>

<!-- Masa SEPARATA - nu se misca -->
<div style="width:220px; height:10px; background:#8B6347; border-radius:4px;
margin-top:-15px;"></div>
<div style="display:flex; justify-content:space-between; width:180px;">
    <div style="width:10px; height:20px; background:#6B4423; border-
radius:2px;"></div>
    <div style="width:10px; height:20px; background:#6B4423; border-
radius:2px;"></div>
</div>
<!-- Status label - mai jos -->
<div id="sl" style="margin-top:16px; padding:5px 18px; border-radius:20px;
    font-size:13px; font-weight:500; background:#2D7A2D; color:white;">
    Loading...
</div>

<!-- Procentele vizuale -->
<div style="display:flex; gap:12px; margin-top:10px; font-size:11px;">
    <span id="pg" style="color:#4CAF50;">● </span>
    <span id="py" style="color:#FFC107;">● </span>
    <span id="pb" style="color:#795548;">● </span>
</div>
</div>
<style>
@keyframes sway {
    0% {
        transform: rotate(-1.5deg);
    }
    50% {
        transform: rotate(1.5deg);

```

```

    }
    100% {
        transform: rotate(-1.5deg);
    }
}
</style>
<script>
(function($scope) {
    $scope.$watch('msg', function(msg) {
        if (!msg) return;
        var stare = msg.stare || 'fericita';
        var culoare = msg.culoare || '#2D7A2D';
        var vizual = msg.vizual || { verde:0, galben:0, maro:0 };
        document.querySelectorAll('.lf').forEach(function(l) {
            l.style.fill = stare === 'fericita' ? l.getAttribute('fill') : culoare;
        });
        document.getElementById('bd').style.fill = culoare;
        var mo = document.getElementById('mo');
        if (stare === 'fericita') mo.setAttribute('d','M52,165 Q60,171 68,165');
        else if (stare === 'neutra') mo.setAttribute('d','M52,166 Q60,166 68,166');
        else mo.setAttribute('d','M52,168 Q60,163 68,168');
        var labels = {
            'fericita': 'Plant is healthy!',
            'neutra': 'Acceptable conditions',
            'trista': 'Plant needs help!'
        };
        var sl = document.getElementById('sl');
        sl.textContent = labels[stare] || stare;
        sl.style.background = culoare;
        document.getElementById('pg').textContent = '  ' + (vizual.verde ||
0).toFixed(1) + '%';
        document.getElementById('py').textContent = '  ' + (vizual.galben ||
0).toFixed(1) + '%';
        document.getElementById('pb').textContent = '  ' + (vizual.maro ||
0).toFixed(1) + '%';

```

```

    });
  })(scope);
</script>

```

3. Codul din Node-RED pentru termometru

```

<div style="
  width: 100%;
  height: 100%;
  padding: 12px;
  box-sizing: border-box;
  background: #222222;
  border: 1px solid #2D4A2D;
  border-radius: 12px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.3);
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;">
  <div style="font-size:18px; color:white; font-weight:500; margin-bottom:12px; text-align:center;">Temperature</div>
  <svg viewBox="0 0 60 160" width="70" height="160">
    <rect x="22" y="10" width="16" height="105" rx="8" fill="#2a2a3e" stroke="#555"
stroke-width="1.5" />
    <rect id="tf" x="24" y="80" width="12" height="35" rx="4" fill="#2ecc71" />
    <circle id="tb" cx="30" cy="118" r="14" fill="#2ecc71" stroke="#555" stroke-
width="1.5" />
    <line x1="38" y1="20" x2="44" y2="20" stroke="#888" stroke-width="1" />
    <line x1="38" y1="35" x2="44" y2="35" stroke="#888" stroke-width="1" />
    <line x1="38" y1="50" x2="44" y2="50" stroke="#888" stroke-width="1" />
    <line x1="38" y1="65" x2="44" y2="65" stroke="#888" stroke-width="1" />
    <line x1="38" y1="80" x2="44" y2="80" stroke="#888" stroke-width="1" />
    <line x1="38" y1="95" x2="44" y2="95" stroke="#888" stroke-width="1" />
    <line x1="38" y1="110" x2="44" y2="110" stroke="#888" stroke-width="1" />
    <text x="46" y="23" font-size="8" fill="#aaa">50°</text>
    <text x="46" y="53" font-size="8" fill="#aaa">30°</text>
    <text x="46" y="83" font-size="8" fill="#aaa">15°</text>

```



```

border: 1px solid #2D4A2D;
border-radius: 12px;
box-shadow: 0 4px 6px rgba(0,0,0,0.3);
display: flex;
flex-direction: column;
justify-content: center;
align-items: center;">
<div style="font-size:18px; color:white; font-weight:500; margin-bottom:8px; text-align:center;">Soil Moisture</div>
<svg viewBox="0 0 120 120" width="160" height="160">
  <defs>
    <clipPath id="circle-clip">
      <circle cx="60" cy="60" r="50" />
    </clipPath>
  </defs>
  <circle cx="60" cy="60" r="50" fill="#1a1a2e" stroke="#444" stroke-width="2" />
  <g clip-path="url(#circle-clip)">
    <rect id="water-bg" x="10" y="115" width="100" height="5" fill="#1a9fd4"
opacity="0.4" />
    <path id="water-wave" d="M10,115 L120,115 L120,115 L10,115 Z"
fill="#1a9fd4" opacity="0.8">
      <animateTransform attributeName="transform" type="translate" from="-50,0"
to="0,0" dur="2s"
repeatCount="indefinite" />
    </path>
  </g>
  <text id="moisture-val" x="60" y="65" text-anchor="middle" font-size="22" font-weight="bold" fill="white">
    --%
  </text>
  <text x="60" y="75" text-anchor="middle" font-size="10"
fill="#aaa">moisture</text>
  <circle cx="60" cy="60" r="50" fill="none" stroke="#1a9fd4" stroke-width="2.5" />
</svg>

```

</div>

<script>

```
(function($scope) {  
  $scope.$watch('msg', function(msg) {  
    if (!msg || msg.payload === undefined) return;  
    var val = parseFloat(msg.payload);  
    if (isNaN(val)) return;  
    var waterY = 110 - (val); // 1% = 1px  
    document.getElementById('water-bg').setAttribute('y', waterY);  
    document.getElementById('water-bg').setAttribute('height', 110 - waterY + 10);  
    document.getElementById('water-wave').setAttribute('d',  
      'M10,' + waterY +  
      ' Q30,' + (waterY - 6) + ' 50,' + waterY +  
      ' Q70,' + (waterY + 6) + ' 90,' + waterY +  
      ' Q110,' + (waterY - 6) + ' 120,' + waterY +  
      ' L120,115 L10,115 Z'  
    );  
    var color;  
    if (val < 25) {  
      color = '#e74c3c'; // rosu - uscat  
    } else if (val < 45) {  
      color = '#e8a020'; // portocaliu - scazut  
    } else {  
      color = '#1a9fd4'; // albastru - ok  
    }  
    document.getElementById('water-wave').style.fill = color;  
    document.getElementById('water-bg').style.fill = color;  
    document.querySelector('circle:last-of-type').style.stroke = color;  
  });  
})(scope);
```

</script>

5. Codul din Node-RED pentru umiditate aer

<div style="

width: 100%;

height: 100%;

```

padding: 12px;
box-sizing: border-box;
background: #222222;
border: 1px solid #2D4A2D;
border-radius: 12px;
box-shadow: 0 4px 6px rgba(0,0,0,0.3);
display: flex;
flex-direction: column;
align-items: center;
justify-content: center;">
  <div style="font-size:18px; color:white; font-weight:500; margin-bottom:4px; text-
align:center; width:100%;">
    Air Humidity
  </div>
  <div
    style="position:relative; width:100%; flex:1; display:flex; flex-direction:column;
align-items:center; min-height:0;">
    <canvas id="humidity-canvas" style="width:100%; max-width:220px;
height:110px;"></canvas>
    <div style="position:absolute; bottom:10px; text-align:center;">
      <span id="humidity-val" style="font-size:24px; color:white; font-
weight:bold;">0.0%</span>
      <div style="font-size:11px; color:#888; margin-top:-2px;">rH</div>
    </div>
  </div>
  <div
    style="width:100%; max-width:200px; display:flex; justify-content:space-be-
tween; font-size:10px; color:#666; margin-top:-5px; padding:0 10px;">
    <span>0</span>
    <span>100</span>
  </div>
</div>
<script>
  (function($scope) {
    var canvas, ctx;

```

```
var currentVal = 0;
function init() {
  canvas = document.getElementById('humidity-canvas');
  if (!canvas) return false;
  ctx = canvas.getContext('2d');
  return true;
}
function draw(val) {
  if (!ctx) return;
  var W = canvas.offsetWidth;
  var H = canvas.offsetHeight;
  canvas.width = W;
  canvas.height = H;
  ctx.clearRect(0, 0, W, H);
  var cx = W / 2;
  var cy = H - 5;
  var radius = Math.min(cx, cy) - 15;
  ctx.beginPath();
  ctx.arc(cx, cy, radius, Math.PI, 0, false);
  ctx.lineWidth = 14;
  ctx.strokeStyle = '#333333';
  ctx.lineCap = 'round';
  ctx.stroke();
  if (val > 0) {
    var angle = Math.PI + (Math.min(val, 100) / 100) * Math.PI;
    ctx.beginPath();
    ctx.arc(cx, cy, radius, Math.PI, angle, false);
    ctx.lineWidth = 14;
    ctx.strokeStyle = '#0096FF'; // Culoarea albastră
    ctx.lineCap = 'round';
    ctx.stroke();
  }
}
$scope.$watch('msg', function(msg) {
  if (!msg || msg.payload === undefined) return;
```

```

var val = parseFloat(msg.payload);
if (isNaN(val)) return;
currentVal = val;
var txt = document.getElementById('humidity-val');
if (txt) txt.textContent = val.toFixed(1) + '%';
if (!ctx && !init()) return;
draw(currentVal);
});
window.addEventListener('resize', function() {
  if (ctx) draw(currentVal);
});
})(scope);
</script>

```

6. Codul din Node-RED pentru luminozitate

```

<div style="
width: 100%;
height: 100%;
padding: 12px;
box-sizing: border-box;
background: #222222;
border: 1px solid #2D4A2D;
border-radius: 12px;
box-shadow: 0 4px 6px rgba(0,0,0,0.3);
display: flex;
flex-direction: column;">
  <div style="font-size:18px; color:white; font-weight:500; margin-bottom:8px; text-align:center; width:100%;">
    Brightness (24h - Logarithmic)</div>
  <div style="display:flex; flex:1; min-height:0;">
    <div
      style="display:flex;      flex-direction:column;      justify-content:space-between;
padding:4px 6px 4px 0; font-size:16px;">
      <span title="Direct Sunlight (50000 lx)">☀️</span>
      <span title="Bright Daylight (10000 lx)">☀️</span>
      <span title="Overcast / Room (1000 lx)">☁️</span>
    </div>
  </div>
</div>

```

```

    <span title="Dim Light (100 lx)">☁</span>

    <span title="Night / Darkness (0-10 lx)">🌙</span>
</div>

<canvas id="lux-canvas" style="flex:1; min-width:0;"></canvas>
</div>
<div id="x-axis"
    style="display:flex; justify-content:space-between; font-size:9px; color:#666;
padding-left:30px; margin-top:2px;">
    </div>
</div>
<script>
(function($scope) {
var history = [];
var MAX = 288;
var MAX_LUX = 50000; // Domeniu extins, compatibil cu BH1750
var canvas, ctx;
function init() {
    canvas = document.getElementById('lux-canvas');
    if (!canvas) return false;
    ctx = canvas.getContext('2d');
    return true;
}
// Functie pentru calcularea inaltimii pe axa Y folosind scara logaritmica
function getLogY(val, H) {
    var safeVal = Math.max(0, Math.min(val, MAX_LUX));
    // ln(val + 1) previne eroarea logaritmului din 0
    var logNorm = Math.log(safeVal + 1) / Math.log(MAX_LUX + 1);
    return H - (logNorm * H);
}

function draw() {
    if (!ctx) return;
    var W = canvas.offsetWidth;
    var H = canvas.offsetHeight;

```

```

canvas.width = W;
canvas.height = H;
ctx.clearRect(0, 0, W, H);
ctx.strokeStyle = '#333';
ctx.lineWidth = 0.5;
var gridLevels = [10, 100, 1000, 10000];
gridLevels.forEach(function(lvl) {
    var gy = getLogY(lvl, H);
    ctx.beginPath();
    ctx.moveTo(0, gy);
    ctx.lineTo(W, gy);
    ctx.stroke();
});
if (history.length < 2) return;
var step = W / (MAX - 1);
var grad = ctx.createLinearGradient(0, 0, 0, H);
grad.addColorStop(0, 'rgba(239,159,39,0.6)');
grad.addColorStop(1, 'rgba(239,159,39,0)');
ctx.beginPath();
history.forEach(function(pt, i) {
    var x = i * step;
    var y = getLogY(pt.v, H);
    if (i === 0) ctx.moveTo(x, H);
    ctx.lineTo(x, y);
});
ctx.lineTo((history.length - 1) * step, H);
ctx.closePath();
ctx.fillStyle = grad;
ctx.fill();
ctx.beginPath();
ctx.strokeStyle = '#EF9F27';
ctx.lineWidth = 2;
history.forEach(function(pt, i) {
    var x = i * step;
    var y = getLogY(pt.v, H);

```

```

        if (i === 0) ctx.moveTo(x, y);
        else ctx.lineTo(x, y);
    });
    ctx.stroke();
    var last = history[history.length - 1];
    var lx = (history.length - 1) * step;
    var ly = getLogY(last.v, H);
    ctx.beginPath();
    ctx.arc(lx, ly, 4, 0, Math.PI * 2);
    ctx.fillStyle = '#EF9F27';
    ctx.fill();
    var xAxis = document.getElementById('x-axis');
    xAxis.innerHTML = "";
    var shown = [];
    var stepLabels = Math.max(1, Math.floor(history.length / 5));
    for (var i = 0; i < history.length; i += stepLabels) {
        shown.push(history[i].t);
    }
    if (shown.length > 0) xAxis.textContent = shown.join(' ');
}

$scope.$watch('msg', function(msg) {
    if (!msg || msg.payload === undefined) return;
    var val = parseFloat(msg.payload);
    if (isNaN(val)) return;
    if (!ctx && !init()) return;
    var now = new Date();
    history.push({
        v: val,
        t: String(now.getHours()).padStart(2, '0') + ':' +
String(now.getMinutes()).padStart(2, '0')
    });
    if (history.length > MAX) history.shift();
    draw();
});
})(scope);

```

</script>

7. Codul din Node-RED pentru asistentul virtual

```
<div id="va-box" style="
width: 100%;
height: 100%;
padding: 12px;
box-sizing: border-box;
background: #222222;
border: 1px solid #2D4A2D;
border-radius: 12px;
box-shadow: 0 4px 6px rgba(0,0,0,0.3);
display: flex;
flex-direction: column;
justify-content: flex-start;
gap: 8px;">
  <div style="display:flex; align-items:center; gap:8px;">
    <span style="font-size:18px; color:white; font-weight:500; margin-bottom:8px;
width:100%;">
      Virtual Assistant
    </span>
    <span id="va-badge" style="
font-size:10px;
padding:4px 12px;
border-radius:20px;
background:#2D7A2D;
color:white;
font-weight:600;
flex-shrink:0;
white-space:nowrap;">
      Healthy
    </span>
  </div>
  <div id="va-msg" style="
font-size:12px;
color:#e0e0e0;
```

```
    line-height:1.6;
    font-weight:400;
    flex-grow: 1;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    gap: 6px;
    overflow: hidden;
    margin-left: 4px;">
    Loading...
</div>
</div>
<script>
(function($scope) {
$scope.$watch('msg', function(msg) {
    if (!msg || !msg.payload) return;
    var text = msg.payload;
    var stare = msg.stare || 'fericita';
    var box = document.getElementById('va-box');
    var badge = document.getElementById('va-badge');
    var msgEl = document.getElementById('va-msg');
    var config = {
        'fericita': {
            badge: '#2D7A2D',
            label: 'Healthy'
        },
        'neutra': {
            badge: '#C8A000',
            label: 'Attention'
        },
        'trista': {
            badge: '#e74c3c',
            label: 'Critical'
        }
    };
```

```

var messages = {
  'Planta este sanatoasa! Continua ingrijirea normala.': {
    title: 'Plant is Healthy!',
    recommendations: [
      '✓ Continue current watering schedule',
      '✓ Maintain optimal light exposure',
      '✓ Monitor temperature stability'
    ]
  },
  'Planta arata sanatoasa! Senzori si camera confirma.': {
    title: 'All Systems Optimal!',
    recommendations: [
      '✓ Sensors confirm perfect conditions',
      '✓ No action required at this time',
      '✓ Growth trajectory is excellent'
    ]
  },
  'Solul incepe sa se usuce. Uda in curand.': {
    title: 'Soil Getting Dry',
    recommendations: [
      '• Water plant within 24 hours',
      '• Check soil depth before watering',
      '• Upload photo to verify plant health'
    ]
  },
  'Uda urgent! Solul este prea uscat.': {
    title: 'Urgent: Water Now!',
    recommendations: [
      '• Water plant immediately',
      '• Soil moisture is critically low',
      '• Upload photo to assess overall health'
    ]
  },
}

```

```
'Prea mult soare! Muta intr-un loc umbros.': {
  title: 'Too Much Sunlight',
  recommendations: [
    '• Move plant to partial shade',
    '• Reduce direct light exposure',
    '• Upload photo to check leaf damage'
  ]
},
'Temperatura prea scazuta. Muta planta la caldura.': {
  title: 'Temperature Too Low',
  recommendations: [
    '• Move to warmer location (18°C+)',
    '• Avoid cold drafts and windows',
    '• Upload photo to monitor plant stress'
  ]
},
'Canicula! Muta planta la racoare.': {
  title: 'EMERGENCY: Extreme Heat!',
  recommendations: [
    '• Move plant to cool, shaded area NOW',
    '• Increase watering frequency',
    '• Upload photo to assess critical damage'
  ]
},
'Planta arata rau': {
  title: 'CRITICAL: Plant Struggling!',
  recommendations: [
    '• Move to shade and water immediately',
    '• Increase humidity around plant',
    '• Upload photo for AI health assessment'
  ]
}
};
var finalMsg = null;
Object.keys(messages).forEach(function(key) {
```

```

    if (text.indexOf(key) !== -1 || key.indexOf(text) !== -1) {
      finalMsg = messages[key];
    }
  }
  if (finalMsg) {
    if (finalMsg.title.indexOf('Healthy') !== -1 || finalMsg.title.indexOf('Optimal') !== -
1) {
      stare = 'fericita';
    } else if (finalMsg.title.indexOf('EMERGENCY') !== -1 ||
finalMsg.title.indexOf('CRITICAL') !== -1) {
      stare = 'trista';
    } else {
      stare = 'neutra';
    }
  }
  var cfg = config[stare] || config['fericita'];
  badge.style.background = cfg.badge;
  badge.textContent = cfg.label;
  if (finalMsg) {
    msgEl.innerHTML = '<strong style="color:#1abc9c; font-size:13px; margin-
bottom:2px;">' + finalMsg.title + '</strong>' +
      finalMsg.recommendations.map(r => '<div>' + r + '</div>').join("");
  } else {
    msgEl.textContent = text;
  }
});
})(scope);
</script>

```

8. Codul din Node-RED pentru transmisia datelor din MATLAB

```

let data = msg.payload;
flow.set("real_vizual", {
  verde: data.percentGreen,
  galben: data.percentYellow,
  maro: data.percentBrown
});

```

```

flow.set("plantHealthStatus", data.status);
msg.payload = data;
return msg;
9. Codul din MATLAB
[fname, fpath] = uigetfile({'*.jpg;*.png;*.jpeg;*.bmp;*.tiff;*.gif'}, 'Select the plant
image for analysis');
if isequal(fname,0) || isequal(fpath,0)
    disp('Selection cancelled.');
```

return;

```

end
I = imread(strcat(fpath,fname));
fprintf('Processing image: %s\n', fname);
rng(42);
figure(1); imshow(I); title('Original Image');
hsv_full = rgb2hsv(I);
S_full = hsv_full(:,:,2);
V_full = hsv_full(:,:,3);
plant_mask = (S_full >= 0.30) & (V_full >= 0.10);
R_all = double(I(:,:,1));
G_all = double(I(:,:,2));
B_all = double(I(:,:,3));
R = R_all(plant_mask);
G = G_all(plant_mask);
B = B_all(plant_mask);
fprintf('Plant pixels kept after background removal: %d / %d\n', length(R),
numel(plant_mask));
NC = 6;
V = R*256^2 + G*256 + B;
culori = sort(V);
culori = culori(diff(culori) > 0);
fprintf('Distinct colors found: %d\n', length(culori));
poz = randperm(length(culori), NC);
VC = culori(poz);
RC = floor(VC/256^2);    VC = VC - RC*256^2;
GC = floor(VC/256);    VC = VC - GC*256;

```

```

BC = VC;
C = zeros(length(R),1);
iter = 0;
while true
    dist = zeros(length(R), NC);
    for k = 1:NC
        dist(:,k) = sqrt((R-RC(k)).^2 + (G-GC(k)).^2 + (B-BC(k)).^2);
    end
    [~, idx] = min(dist, [], 2);
    C = idx;
    new_RC = zeros(NC,1); new_GC = zeros(NC,1); new_BC = zeros(NC,1);
    for k = 1:NC
        len = sum(C==k);
        if len == 0
            new_RC(k) = RC(k); new_GC(k) = GC(k); new_BC(k) = BC(k);
            continue;
        end
        new_RC(k) = round(sum((C==k).*R)/len);
        new_GC(k) = round(sum((C==k).*G)/len);
        new_BC(k) = round(sum((C==k).*B)/len);
    end
    iter = iter+1;
    dif = sum(abs(new_RC-RC)) + sum(abs(new_GC-GC)) + sum(abs(new_BC-BC));
    fprintf('Iter: %d, centroid difference: %d\n', iter, dif);
    RC = new_RC; GC = new_GC; BC = new_BC;
    if dif < NC || iter > 50
        break;
    end
end
lin = size(I,1); col = size(I,2);
new_I = zeros(lin, col, 3, 'uint8');
full_C = zeros(lin*col,1);
full_C(plant_mask) = C;
full_C = reshape(full_C, lin, col);

```

```

for k = 1:NC
    mask_k = (full_C == k);
    new_I(:, :, 1) = new_I(:, :, 1) + uint8(mask_k)*uint8(RC(k));
    new_I(:, :, 2) = new_I(:, :, 2) + uint8(mask_k)*uint8(GC(k));
    new_I(:, :, 3) = new_I(:, :, 3) + uint8(mask_k)*uint8(BC(k));
end
figure(2); imshow(new_I); title(sprintf('K-means Segmentation (background
removed, %d clusters)', NC));
hsv_centroids = rgb2hsv([RC GC BC]/255);
hues = hsv_centroids(:,1);
sats = hsv_centroids(:,2);
vals = hsv_centroids(:,3);
label = cell(NC,1);
fprintf('\nCentroid colors found (RGB -> HSV):\n');
for k = 1:NC
    h = hues(k); s = sats(k); v = vals(k);
    fprintf('Cluster %d: R=%d G=%d B=%d -> H=%.2f S=%.2f V=%.2f\n', k, RC(k),
GC(k), BC(k), h, s, v);
    if h >= 0.16
        label{k} = 'green';
    elseif h >= 0.13 && h < 0.16
        label{k} = 'yellow';
    elseif h >= 0.02 && h < 0.13
        label{k} = 'brown';
    else
        label{k} = 'other';
    end
end
end
pixels_green = 0; pixels_yellow = 0; pixels_brown = 0;
for k = 1:NC
    count_k = sum(C==k);
    switch label{k}
        case 'green', pixels_green = pixels_green + count_k;
        case 'yellow', pixels_yellow = pixels_yellow + count_k;
        case 'brown', pixels_brown = pixels_brown + count_k;
    end
end

```

```

        end
    end
    fprintf('\nCluster labels: ');
    fprintf('%s ', label{:});
    fprintf('\n');
    total_plant_pixels = pixels_green + pixels_yellow + pixels_brown;
    if total_plant_pixels == 0
        fprintf('\n[WARNING] No cluster matched plant colors. Try a different image or
        NC.\n');
        return;
    end
    percent_green = (pixels_green / total_plant_pixels) * 100;
    percent_yellow = (pixels_yellow / total_plant_pixels) * 100;
    percent_brown = (pixels_brown / total_plant_pixels) * 100;
    fprintf('\n--- PLANT HEALTH ANALYSIS RESULTS (K-means) ---\n');
    fprintf('GREEN: %.2f%%\n', percent_green);
    fprintf('YELLOW: %.2f%%\n', percent_yellow);
    fprintf('BROWN: %.2f%%\n', percent_brown);
    fprintf('-----\n');
    if percent_green >= 60
        status = 'Healthy';
    elseif percent_green >= 35
        status = 'Warning';
    else
        status = 'Critical';
    end
    plantData.percentGreen = percent_green;
    plantData.percentYellow = percent_yellow;
    plantData.percentBrown = percent_brown;
    plantData.status = status;
    url = 'http://172.20.10.2:1880/plant-health';
    options = weboptions('MediaType', 'application/json', 'Timeout', 10);
    try
        webwrite(url, plantData, options);
        fprintf('\n[OK] Data sent to Node-RED successfully.\n');
    end
end

```

```

    fprintf('Status: %s\n', status);
catch ME
    fprintf('\n[ERROR] Failed to send data to Node-RED: %s\n', ME.message);
end
10. Codul din Arduino
#include "DHT.h"
#include <Wire.h>
#include <BH1750.h>
#include <WiFi.h>
#include <PubSubClient.h>
const char* ssid = "ESP32Test";
const char* password = "12345678";
const char* mqtt_server = "172.20.10.2";
const int mqtt_port = 1883;
WiFiClient espClient;
PubSubClient client(espClient);
#define DHTPIN 23
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);
#define SOIL_PIN 34
BH1750 lightMeter;
void setup_wifi() {
    Serial.println();
    Serial.print("Conectare la WiFi: ");
    Serial.println(ssid);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("");
    Serial.println("WiFi conectat!");
    Serial.print("Adresa IP ESP32: ");
    Serial.println(WiFi.localIP());
}

```

```

void reconnect_mqtt() {
  while (!client.connected()) {
    Serial.print("Conectare la broker MQTT...");
    if (client.connect("ESP32SmartGarden")) {
      Serial.println("conectat!");
    } else {
      Serial.print("eroare, rc=");
      Serial.print(client.state());
      Serial.println(" reincerc in 5 secunde");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  Serial.println("=====
==");
  Serial.println("  SISTEM IOT REAL: AER + SOL CAPACITIV + LUMINĂ +
MQTT");
  Serial.println("=====
==");
  dht.begin();
  Wire.begin(21, 22);
  if (lightMeter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE)) {
    Serial.println("[OK] Senzorul de lumină BH1750 a pornit cu succes!");
  } else {
    Serial.println("[EROARE] Nu am găsit senzorul BH1750! Verifică firele P21/P22.");
  }

  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
}

void loop() {
  if (!client.connected()) {

```

```

    reconnect_mqtt();
}
client.loop();
delay(2000);
float umiditateAer = dht.readHumidity();
float temperaturaAer = dht.readTemperature();
int valoareSolBruta = analogRead(SOIL_PIN);
float luminaLux = lightMeter.readLightLevel();
if (isnan(umiditateAer) || isnan(temperaturaAer)) {
    Serial.println("[EROARE] Senzorul DHT22 nu trimite date! Verifică firele.");
    return;
}
Serial.print("AER -> Temp: ");
Serial.print(temperaturaAer, 1);
Serial.print("°C | Umiditate: ");
Serial.print(umiditateAer, 1);
Serial.print("% || SOL -> Brut: ");
Serial.print(valoareSolBruta);
Serial.print(" || LUMINĂ -> ");
Serial.print(luminaLux, 1);
Serial.println(" lux");
char buffer[10];
dtostrf(temperaturaAer, 4, 1, buffer);
client.publish("smartgarden/temperatura", buffer);
dtostrf(umiditateAer, 4, 1, buffer);
client.publish("smartgarden/umiditate_aer", buffer);
dtostrf(luminaLux, 6, 1, buffer);
client.publish("smartgarden/lumina", buffer);
itoa(valoareSolBruta, buffer, 10);
client.publish("smartgarden/sol", buffer);
}

```
